PROYEK AKHIR SEMESTER GASAL

NAMA : Noval Akbar
KELAS : X PPLG 1
NIS : 258725

JUDUL PROYEK : APLIKASI CUACA

PENGEMBANGAN PERANGKAT LUNAK DAN GIM

SMK NEGERI 1 KANDEMAN

TAHUN PELAJARAN 2025/2026

DESKRIPSI PROYEK

Proyek ini bertujuan untuk membuat **Aplikasi Cuaca** berbasis desktop menggunakan C# Windows Forms yang dapat menampilkan informasi cuaca real-time dari berbagai lokasi di seluruh dunia. Aplikasi ini mengintegrasikan **WeatherAPI** untuk mengambil data cuaca terkini dan ramalan cuaca beberapa hari ke depan.

Fitur Utama Aplikasi:

1. **Pencarian Lokasi**
    - Pengguna dapat memasukkan nama kota untuk mencari informasi cuaca
    - Mendukung pencarian kota di seluruh dunia
2. **Informasi Cuaca Saat Ini**
    - Menampilkan suhu dalam Celsius
    - Kondisi cuaca (cerah, berawan, hujan, dll)
    - Suhu yang dirasakan (feels like)
    - Icon cuaca yang dinamis sesuai kondisi dan waktu (siang/malam)
3. **Data Detail Cuaca**
    - **Indeks UV**: Tingkat radiasi ultraviolet
    - **Kelembaban**: Persentase kelembaban udara
    - **Tekanan Udara**: Dalam millibar (mb)
    - **Tutupan Awan**: Persentase awan
    - **Kecepatan Angin**: Dalam km/h beserta arah angin
    - **Curah Hujan**: Dalam millimeter (mm)
4. **Informasi Kualitas Udara**
    - Indeks kualitas udara (Air Quality Index)
    - PM 2.5 (partikel halus)
    - PM 10 (partikel kasar)
    - Carbon Monoxide (CO)
    - Ozone (O3)
5. **Ramalan Cuaca 3 Hari**
    - Tanggal ramalan
    - Suhu maksimal dan minimal
    - Kondisi cuaca
    - Icon cuaca untuk setiap hari
6. **Icon Dinamis**
    - Icon berubah otomatis berdasarkan kondisi cuaca

- Perbedaan icon untuk siang dan malam hari
- Icon indikator untuk berbagai parameter cuaca

Konsep Pemrograman yang Diimplementasikan:

1. **Async/Await Programming** - Untuk operasi asynchronous saat mengambil data dari API
2. **HTTP Client** - Untuk melakukan request ke WeatherAPI
3. **JSON Deserialization** - Parsing data JSON dari API menggunakan Newtonsoft.Json
4. **Class dan Object** - Membuat class model untuk data cuaca (Root, Current, Location, dll)
5. **Dictionary** - Mapping kode cuaca ke nama icon
6. **GUI (Graphical User Interface)** - Antarmuka visual menggunakan Windows Forms
7. **Error Handling** - Try-catch untuk menangani error
8. **Event Handling** - Menangani event button click dan key press
9. **Static Class dan Method** - Class WeatherIcon sebagai utility class
10. **LINQ dan Collections** - Menggunakan List dan Dictionary untuk menyimpan data

RINGKASAN TEORI C# YANG DIGUNAKAN

1. Async/Await Programming

Async/await digunakan untuk operasi yang membutuhkan waktu (seperti request ke API) agar aplikasi tidak freeze.

```
private async Task GetWeatherData(string city)
{
     HttpResponseMessage response = await httpClient.GetAsync(url);
     string json = await response.Content.ReadAsStringAsync();
}
```

Penjelasan:

- async - Menandai method sebagai asynchronous
- await - Menunggu operasi selesai tanpa memblokir UI
- Task - Representasi operasi asynchronous

2. HTTP Client

HttpClient digunakan untuk berkomunikasi dengan web API.

```
private HttpClient httpClient;
httpClient = new HttpClient();
HttpResponseMessage response = await httpClient.GetAsync(url);
```

Komponen:

- HttpClient - Class untuk mengirim HTTP request
- GetAsync() - Method untuk HTTP GET request
- HttpResponseMessage - Response dari server

3. JSON Deserialization

Mengubah data JSON dari API menjadi object C#.

```
string json = await response.Content.ReadAsStringAsync();
Root weatherData = JsonConvert.DeserializeObject<Root>(json)!;
```

Library: Newtonsoft.Json (Json.NET)

4. Class dan Object

Membuat struktur data untuk merepresentasikan data cuaca.

```
public class Current
{
    public required string last_updated { get; set; }
    public required object temp_c { get; set; }
    public required int is_day { get; set; }
    public required Condition condition { get; set; }
    public required double wind_kph { get; set; }
    public required string wind_dir { get; set; }
    public required double pressure_mb { get; set; }
    public required double precip_mm { get; set; }
    public required double humidity { get; set; }
    public required double cloud { get; set; }
    public required double feelslike_c { get; set; }
    public required double uv { get; set; }
    public required double gust_mph { get; set; }
    public required object gust_kph { get; set; }
    public required AirQuality air_quality { get; set; }
}
```

Konsep:

- required - Property wajib diisi
- { get; set; } - Auto-implemented property

5. Dictionary

Menyimpan mapping antara kode cuaca dengan nama icon.

```
private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int,
string>();
{
    // Cerah/Berawan
    { 1000, "clear-day" },

    // Cerah Berawan
    { 1003, "partly-cloudy-day" },

    // Berawan
    { 1006, "cloudy" },
    { 1009, "overcast" },

    // Kabut
    { 1030, "mist" },
    { 1135, "fog" },
    { 1147, "fog" },
```

```
    // Gerimis
    { 1063, "drizzle" },
    { 1150, "drizzle" },
    { 1153, "drizzle" },
    { 1168, "sleet" },
    { 1171, "sleet" },

    // Hujan
    { 1180, "rain" },
    { 1183, "rain" },
    { 1186, "rain" },
    { 1189, "rain" },
    { 1192, "rain" },
    { 1195, "rain" },
    { 1198, "sleet" },
    { 1201, "sleet" },
    { 1240, "rain" },
    { 1243, "rain" },
    { 1246, "rain" },

    // Salju
    { 1066, "snow" },
    { 1069, "sleet" },
    { 1072, "sleet" },
    { 1114, "snow" },
    { 1117, "snow" },
    { 1204, "sleet" },
    { 1207, "sleet" },
    { 1210, "snow" },
    { 1213, "snow" },
    { 1216, "snow" },
    { 1219, "snow" },
    { 1222, "snow" },
    { 1225, "snow" },
    { 1237, "hail" },
    { 1249, "sleet" },
    { 1252, "sleet" },
    { 1255, "snow" },
    { 1258, "snow" },
    { 1261, "hail" },
    { 1264, "hail" },

    // Badai Petir
    { 1087, "thunderstorms-day" },
    { 1273, "thunderstorms-day-rain" },
    { 1276, "thunderstorms-rain" },
    { 1279, "thunderstorms-day-snow" },
    { 1282, "thunderstorms-snow" }
};
```

6. Switch Expression

Menentukan deskripsi kualitas udara berdasarkan indeks.

```
private string GetAQIDescription(int index)
{
    return index switch
    {
        1 => "Good", // Baik
        2 => "Moderate", // Sedang
        3 => "Unhealthy for Sensitive Groups", // Tidak sehat untuk kelompok
sensitif
```

```
        4 => "Unhealthy", // Tidak sehat
        5 => "Very Unhealthy", // Sangat tidak sehat
        6 => "Hazardous", // Berbahaya
        _ => "Not Available" // Tidak tersedia
    };

}
```

7. Event Handling

Menangani event dari UI seperti button click.

```
private async void btnSearch_Click(object sender, EventArgs e)
{
  string city = txtCity.Text.Trim();
  await GetWeatherData(city);
}
```

8. Conditional Statements

Menentukan icon yang tepat berdasarkan kondisi.

```
if (data.current.pressure_mb >= 1013.25)
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-high.png"));
}
else
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-low.png"));
}

// Load icon angin (alert jika kecepatan >= 21 km/h)
if (data.current.wind_kph >= 21)
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-
alert.png"));
}
else
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"wind.png"));
}

// Load icon curah hujan (banyak tetes jika >= 11 mm)
if (data.current.precip_mm >= 11)
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrops.png"));
}
else
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrop.png"));
}
```

```
// Load icon UV berdasarkan indeks UV (0-11+)
int uvIndex = (int)Math.Floor(data.current.uv);
uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11

string iconName = uvIndex >= 1
    ? $"uv-index-{uvIndex}.png"
    : "not-available.png";

picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));

// Load icon awan berdasarkan persentase awan
if (data.current.cloud <= 50)
{
    picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"mist.png"));
}
else
{
    picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"overcast.png"));

}
```

9. String Interpolation

Membuat string dinamis dengan mudah.

```
string url =
$"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
```

10. Try-Catch Exception Handling

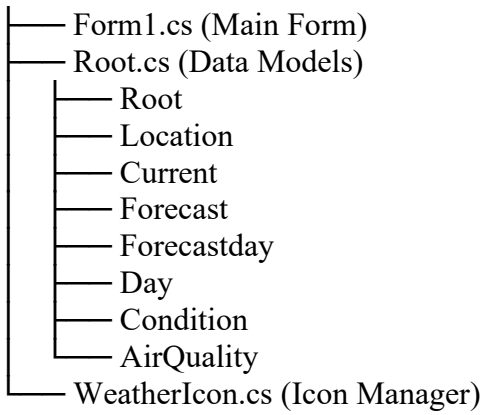
Menangani error yang mungkin terjadi.

```
try
{
  HttpResponseMessage response = await httpClient.GetAsync(url);
  // Kirim request
}
catch (Exception ex)
{
  MessageBox.Show($"Error: {ex.Message}", "Error",
     MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

PERANCANGAN PROGRAM

Struktur Class

WeatherApp



Alur Kerja Program

1. **User Input** → User memasukkan nama kota di TextBox
2. **Validation** → Program mengecek apakah input kosong
3. **API Request** → Program mengirim request ke WeatherAPI
4. **JSON Response** → API mengembalikan data dalam format JSON
5. **Deserialization** → JSON diubah menjadi object C#
6. **Display Data** → Data ditampilkan ke UI (Label, PictureBox)
7. **Icon Loading** → Icon cuaca dimuat dari folder local

API Endpoint

http://api.weatherapi.com/v1/forecast.json?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes

Parameter:

- key - API Key untuk autentikasi
- q - Query lokasi (nama kota)
- days - Jumlah hari forecast (5 hari)
- aqi - Include air quality data (yes)
- alerts - Include weather alerts (yes)

IMPLEMENTASI PROGRAM

1. Form1.cs - Main Application

```
using Newtonsoft.Json;
using System;
using System.Drawing;
using System.Globalization;
using System.IO;
using System.Net.Http;
using System.Threading.Tasks;
using System.Windows.Forms;
using WeatherApp;

namespace WeatherApp
{
    public partial class Form1 : Form
    {
        // Konstanta API
        private const string API_KEY = "xxxxxxxxxxxxxxx";
        private const string FORECAST_API_URL =
"http://api.weatherapi.com/v1/forecast.json";
        private HttpClient httpClient;

        // PictureBox untuk icon
        private PictureBox pictureBoxWeather;
        private PictureBox picDay1, picDay2, picDay3;

        public Form1()
        {
            InitializeComponent();
            httpClient = new HttpClient();
            string iconFolder = Path.Combine(Application.StartupPath, "icons");
            WeatherIcon.IconFolder(iconFolder);
            InitializeWeatherIcons();
        }

        // Inisialisasi PictureBox
        private void InitializeWeatherIcons()
        {
            pictureBoxWeather = new PictureBox();
            pictureBoxWeather.Location = new Point(35, 47);
            pictureBoxWeather.Size = new Size(140, 140);
            pictureBoxWeather.SizeMode = PictureBoxSizeMode.Zoom;
            pictureBoxWeather.BackColor = Color.Transparent;
            panelForecast.Controls.Add(pictureBoxWeather);
            pictureBoxWeather.BringToFront();

            picDay1 = CreateForecastIcon(panelDay1);
            picDay2 = CreateForecastIcon(panelDay2);
            picDay3 = CreateForecastIcon(panelDay3);
        }

        // Helper untuk membuat forecast icon
        private PictureBox CreateForecastIcon(Panel panel)
        {
            var pictureBox = new PictureBox();
            pictureBox.Location = new Point(53, 23);
            pictureBox.Size = new Size(80, 80);
            pictureBox.SizeMode = PictureBoxSizeMode.Zoom;
            pictureBox.BackColor = Color.Transparent;
            panel.Controls.Add(pictureBox);
            panel.BringToFront();
```

```
            return pictureBox;
        }

        // Event handler tombol Search
        private async void btnSearch_Click(object sender, EventArgs e)
        {
            string city = txtCity.Text.Trim();

            if (string.IsNullOrEmpty(city))
            {
                MessageBox.Show("Please enter a city name!", "Warning",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            btnSearch.Enabled = false;
            btnSearch.Text = "Loading...";
            lblStatus.Text = "Fetching weather data...";
            lblStatus.ForeColor = Color.White;

            await GetWeatherData(city);

            btnSearch.Enabled = true;
            btnSearch.Text = "Search";
        }

        // Fungsi untuk mengambil data dari API
        private async Task GetWeatherData(string city)
        {
            try
            {
                string url =
$"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
                HttpResponseMessage response = await httpClient.GetAsync(url);

                if (response.IsSuccessStatusCode)
                {
                    string json = await response.Content.ReadAsStringAsync();
                    Root weatherData =
JsonConvert.DeserializeObject<Root>(json)!;
                    DisplayWeatherData(weatherData);
                }
                else
                {
                    lblStatus.Text = "City not found!";
                    lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
                    ClearWeatherDisplay();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Error: {ex.Message}", "Error",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
                lblStatus.Text = "Error loading data";
                lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
            }
        }

        // Fungsi untuk menampilkan data ke UI
        private void DisplayWeatherData(Root data)
        {
            if (data == null) return;
```

```
            // Data lokasi
            lblCity.Text = data.location.name;
            lblCountry.Text = $"{data.location.region}, {data.location.country}";
            lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}";

            // Data cuaca saat ini
            lblTemp.Text = $"{data.current.temp_c}°C";
            lblCondition.Text = data.current.condition.text;
            lblFeelsLike.Text = $"Feels like: {data.current.feelslike_c}°C";

            // Data detail
            lblUVValue.Text = data.current.uv.ToString();
            lblHumidityValue.Text = $"{data.current.humidity}%";
            lblPressureValue.Text = $"{data.current.pressure_mb} mb";
            lblCloudValue.Text = $"{data.current.cloud}%";
            lblWindValue.Text = $"{data.current.wind_kph}
km/h\n{data.current.wind_dir}";
            lblPrecipValue.Text = $"{data.current.precip_mm} mm";

            // Data kualitas udara
            if (data.current.air_quality != null)
            {
                lblAQValue.Text = $"Air Quality
\n{GetAQIDescription(data.current.air_quality.usepaindex)}";
                lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1}
µg/m³";
                lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1}
µg/m³";
                lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
                lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
            }

            // Load icon cuaca
            LocalIcon(data.current.condition.code, data.current.is_day == 1,
pictureBoxWeather);

            // Tampilkan forecast 3 hari
            if (data.forecast.forecastday != null &&
data.forecast.forecastday.Count >= 3)
            {
                DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date,
lblDay1Temp, lblDay1Condition, picDay1);
                DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date,
lblDay2Temp, lblDay2Condition, picDay2);
                DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date,
lblDay3Temp, lblDay3Condition, picDay3);
            }

            // Load icon detail cuaca
            LoadDetailIcons(data);

            lblStatus.Text = "Weather data loaded successfully!";
            lblStatus.ForeColor = Color.FromArgb(150, 255, 150);
        }

        // Fungsi untuk menampilkan forecast per hari
        private void DisplayForecastDay(Forecastday forecastDay, Label lblDate,
Label lblTemp, Label lblCondition, PictureBox pictureBox)
        {
            if (forecastDay == null) return;

            DateTime date = DateTime.Parse(forecastDay.date);
```

```
            lblDate.Text = date.ToString("ddd, MMM dd",
CultureInfo.InvariantCulture);
            lblTemp.Text =
$"{forecastDay.day.maxtemp_c:F0}°/{forecastDay.day.mintemp_c:F0}°";
            lblCondition.Text = forecastDay.day.condition.text;

            LocalIcon(forecastDay.day.condition.code, true, pictureBox);
        }

        // Fungsi untuk load icon cuaca
        private void LocalIcon(int weatherCode, bool isDay, PictureBox
pictureBox)
        {
            try
            {
                Image icon = WeatherIcon.LoadWeatherIcon(weatherCode, isDay);
                if (icon != null)
                    pictureBox.Image = icon;
                else
                    pictureBox.Image = null;
            }
            catch
            {
                pictureBox.Image = null;
            }
        }

        // Fungsi untuk mendapatkan deskripsi AQI
        private string GetAQIDescription(int index)
        {
            return index switch
            {
                1 => "Good",
                2 => "Moderate",
                3 => "Unhealthy for Sensitive Groups",
                4 => "Unhealthy",
                5 => "Very Unhealthy",
                6 => "Hazardous",
                _ => "Not Available"
            };
        }

        // Fungsi untuk membersihkan tampilan
        private void ClearWeatherDisplay()
        {
            lblCity.Text = "-";
            lblCountry.Text = "-";
            lblTemp.Text = "-";
            lblCondition.Text = "-";
            lblFeelsLike.Text = "Feels like: -";
            // ... reset semua label lainnya
        }

        // Event handler ketika user tekan Enter
        private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
        {
            if (e.KeyChar == (char)Keys.Enter)
            {
                btnSearch_Click(sender, e);
                e.Handled = true;
            }
        }
```

```
        // Cleanup saat form ditutup
        protected override void OnFormClosing(FormClosingEventArgs e)
        {
            httpClient?.Dispose();
            base.OnFormClosing(e);
        }
    }
}
```

2. Root.cs - Data Models

```
using Newtonsoft.Json;

namespace WeatherApp
{
    // Class untuk kualitas udara
    public class AirQuality
    {
        public required object co { get; set; }
        public required object no2 { get; set; }
        public required object o3 { get; set; }
        public required object so2 { get; set; }
        public required object pm2_5 { get; set; }
        public required object pm10 { get; set; }

        [JsonProperty("us-epa-index")]
        public required int usepaindex { get; set; }

        [JsonProperty("gb-defra-index")]
        public required int gbdefraindex { get; set; }
    }

    // Class untuk kondisi cuaca
    public class Condition
    {
        public required string text { get; set; }
        public required string icon { get; set; }
        public required int code { get; set; }
    }

    // Class untuk cuaca saat ini
    public class Current
    {
        public required object last_updated_epoch { get; set; }
        public required string last_updated { get; set; }
        public required object temp_c { get; set; }
        public required object temp_f { get; set; }
        public required int is_day { get; set; }
        public required Condition condition { get; set; }
        public required object wind_mph { get; set; }
        public required double wind_kph { get; set; }
        public required object wind_degree { get; set; }
        public required string wind_dir { get; set; }
        public required double pressure_mb { get; set; }
        public required double pressure_in { get; set; }
        public required double precip_mm { get; set; }
        public required double precip_in { get; set; }
        public required double humidity { get; set; }
        public required double cloud { get; set; }
        public required double feelslike_c { get; set; }
        public required double feelslike_f { get; set; }
        public required double vis_km { get; set; }
```

```
    public required double vis_miles { get; set; }
    public required double uv { get; set; }
    public required double gust_mph { get; set; }
    public required object gust_kph { get; set; }
    public required AirQuality air_quality { get; set; }
}

// Class untuk lokasi
public class Location
{
    public required string name { get; set; }
    public required string region { get; set; }
    public required string country { get; set; }
    public required object lat { get; set; }
    public required object lon { get; set; }
    public required string tz_id { get; set; }
    public required object localtime_epoch { get; set; }
    public required string localtime { get; set; }
}

// Class untuk forecast
public class Forecast
{
    public required List<Forecastday> forecastday { get; set; }
}

// Class untuk forecast per hari
public class Forecastday
{
    public required string date { get; set; }
    public required int date_epoch { get; set; }
    public required Day day { get; set; }
}

// Class untuk detail cuaca harian
public class Day
{
    public required double maxtemp_c { get; set; }
    public required double maxtemp_f { get; set; }
    public required double mintemp_c { get; set; }
    public required double mintemp_f { get; set; }
    public required double avgtemp_c { get; set; }
    public required double avgtemp_f { get; set; }
    public required double maxwind_mph { get; set; }
    public required double maxwind_kph { get; set; }
    public required double totalprecip_mm { get; set; }
    public required double totalprecip_in { get; set; }
    public required double avgvis_km { get; set; }
    public required double avgvis_miles { get; set; }
    public required int avghumidity { get; set; }
    public required int daily_will_it_rain { get; set; }
    public required int daily_chance_of_rain { get; set; }
    public required int daily_will_it_snow { get; set; }
    public required int daily_chance_of_snow { get; set; }
    public required Condition condition { get; set; }
    public required double uv { get; set; }
}

// Class Root sebagai container utama
public class Root
{
    public required Location location { get; set; }
    public required Current current { get; set; }
```

```
        public required Forecast forecast { get; set; }
        public required object alert { get; set; }
    }
}
```

3. WeatherIcon.cs - Icon Manager

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.IO;
using System.Windows.Forms;

namespace WeatherApp
{
    public static class WeatherIcon
    {
        private static Dictionary<int, string> weatherCodeToIcon = new
Dictionary<int, string>();
        private static string iconFolder = Path.Combine(Application.StartupPath,
"icons");

        static WeatherIcon()
        {
            InitializeWeatherCodes();
        }

        public static void IconFolder(string folderPath)
        {
            iconFolder = folderPath;
        }

        // Inisialisasi mapping kode cuaca ke icon
        private static void InitializeWeatherCodes()
        {
            weatherCodeToIcon = new Dictionary<int, string>
            {
                { 1000, "clear-day" },
                { 1003, "partly-cloudy-day" },
                { 1006, "cloudy" },
                { 1009, "overcast" },
                { 1030, "mist" },
                { 1135, "fog" },
                { 1147, "fog" },
                { 1063, "drizzle" },
                { 1150, "drizzle" },
                { 1153, "drizzle" },
                { 1180, "rain" },
                { 1183, "rain" },
                { 1186, "rain" },
                { 1189, "rain" },
                { 1192, "rain" },
                { 1195, "rain" },
                { 1066, "snow" },
                { 1087, "thunderstorms-day" },
                { 1273, "thunderstorms-day-rain" },
                // ... dan lainnya
            };
        }

        // Fungsi untuk load icon cuaca
        public static Image LoadWeatherIcon(int weatherCode, bool isDay)
```

```
        {
            try
            {
                string iconName = GetIconName(weatherCode, isDay);
                string iconPath = Path.Combine(iconFolder, $"{iconName}.png");

                if (File.Exists(iconPath))
                {
                    using (var originalImage = Image.FromFile(iconPath))
                    {
                        return new Bitmap(originalImage);
                    }
                }
                return null!;
            }
            catch (Exception ex)
            {
                System.Diagnostics.Debug.WriteLine($"Error loading icon:
{ex.Message}");
                return null!;
            }
        }

        // Fungsi untuk mendapatkan nama icon
        private static string GetIconName(int weatherCode, bool isDay)
        {
            string iconName;

            if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
            {
                iconName = isDay ? "clear-day" : "clear-night";
            }

            // Ganti icon siang dengan malam
            if (!isDay)
            {
                if (weatherCode == 1000)
                    iconName = "clear-night";
                else if (weatherCode == 1003)
                    iconName = "partly-cloudy-night";
                else if (iconName.Contains("-day"))
                    iconName = iconName.Replace("-day", "-night");
            }

            return iconName;
        }
    }
}
```

PENJELASAN KODE PROGRAM

A. Struktur Utama Form1.cs

1. Deklarasi Konstanta dan Variable

```
private const string API_KEY = "xxxxxxxxxxxxxxxx";
private const string FORECAST_API_URL =
"http://api.weatherapi.com/v1/forecast.json";
private HttpClient httpClient;
```

- API_KEY - Kunci autentikasi untuk mengakses WeatherAPI
- FORECAST_API_URL - URL endpoint API untuk data forecast
- httpClient - Instance untuk melakukan HTTP request

2. Constructor dan Inisialisasi

```
public Form1()
{
    InitializeComponent();
    httpClient = new HttpClient();
    string iconFolder = Path.Combine(Application.StartupPath, "icons");
    WeatherIcon.IconFolder(iconFolder);
    InitializeWeatherIcons();
}
```

Proses:

1. Inisialisasi komponen form
2. Membuat HttpClient baru
3. Menentukan folder icon
4. Menginisialisasi PictureBox untuk icon cuaca

3. Event Handler Button Search

```
private async void btnSearch_Click(object sender, EventArgs e)
{
    string city = txtCity.Text.Trim();

    if (string.IsNullOrEmpty(city))
    {
        MessageBox.Show("Please enter a city name!", "Warning",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    btnSearch.Enabled = false;
    btnSearch.Text = "Loading...";
    await GetWeatherData(city);

    btnSearch.Enabled = true;
    btnSearch.Text = "Search";
}
```

Alur:

1. Ambil input nama kota dari TextBox

2. Validasi input tidak kosong
3. Disable button dan ubah teks jadi "Loading..."
4. Panggil fungsi GetWeatherData (async)
5. Enable kembali button setelah selesai

4. Fungsi GetWeatherData (Async)

```
private async Task GetWeatherData(string city)
{
    try
    {
        string url =
$"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
        HttpResponseMessage response = await httpClient.GetAsync(url);

        if (response.IsSuccessStatusCode)
        {
            string json = await response.Content.ReadAsStringAsync();
            Root weatherData = JsonConvert.DeserializeObject<Root>(json)!;
            DisplayWeatherData(weatherData);
        }
        else
        {
            ClearWeatherDisplay();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error: {ex.Message}", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
```

Proses:

1. Buat URL dengan parameter (API key, city, days, aqi)
2. Kirim HTTP GET request ke API
3. Cek apakah response sukses (status 200)
4. Baca content response sebagai string JSON
5. Deserialize JSON menjadi object Root
6. Tampilkan data ke UI

5. Fungsi DisplayWeatherData

```
private void DisplayWeatherData(Root data)
{
    if (data == null) return;

    lblCity.Text = data.location.name;
    lblCountry.Text = $"{data.location.region}, {data.location.country}";
    lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}";

    lblTemp.Text = $"{data.current.temp_c}°C";
```

```
    lblCondition.Text = data.current.condition.text;
    lblFeelsLike.Text = $"Feels like: {data.current.feelslike_c}°C";

    lblUVValue.Text = data.current.uv.ToString();
    lblHumidityValue.Text = $"{data.current.humidity}%";
    lblPressureValue.Text = $"{data.current.pressure_mb} mb";
    lblCloudValue.Text = $"{data.current.cloud}%";
    lblWindValue.Text = $"{data.current.wind_kph} km/h\n{data.current.wind_dir}";
    lblPrecipValue.Text = $"{data.current.precip_mm} mm";

    if (data.current.air_quality != null)
    {
        lblAQValue.Text = $"Air Quality
\n{GetAQIDescription(data.current.air_quality.usepaindex)}";
        lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1} µg/m³";
        lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1} µg/m³";
        lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
        lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
    }

    LocalIcon(data.current.condition.code, data.current.is_day == 1,
pictureBoxWeather);

    if (data.forecast.forecastday != null && data.forecast.forecastday.Count >=
3)
    {
        DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date,
lblDay1Temp, lblDay1Condition, picDay1);
        DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date,
lblDay2Temp, lblDay2Condition, picDay2);
        DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date,
lblDay3Temp, lblDay3Condition, picDay3);
    }
}
```

Fungsi:

- Mengambil data dari object Root dan menampilkannya ke Label/PictureBox
- Menggunakan string interpolation untuk format teks
- Memanggil LocalIcon() untuk memuat icon cuaca
- Menampilkan forecast 3 hari dengan looping

B. Class WeatherIcon.cs

1. Dictionary Mapping

```
private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int,
string>();
{
    { 1000, "clear-day" },
    { 1003, "partly-cloudy-day" },
    { 1006, "cloudy" },
    { 1009, "overcast" },
    { 1030, "mist" },
    { 1135, "fog" },
    { 1147, "fog" },
    { 1063, "drizzle" },
    { 1150, "drizzle" },
    { 1153, "drizzle" },
    { 1168, "sleet" },
    { 1171, "sleet" },
    { 1180, "rain" },
    { 1183, "rain" },
```

```
    { 1186, "rain" },
    { 1189, "rain" },
    { 1192, "rain" },
    { 1195, "rain" },
    { 1198, "sleet" },
    { 1201, "sleet" },
    { 1240, "rain" },
    { 1243, "rain" },
    { 1246, "rain" },
    { 1066, "snow" },
    { 1069, "sleet" },
    { 1072, "sleet" },
    { 1114, "snow" },
    { 1117, "snow" },
    { 1204, "sleet" },
    { 1207, "sleet" },
    { 1210, "snow" },
    { 1213, "snow" },
    { 1216, "snow" },
    { 1219, "snow" },
    { 1222, "snow" },
    { 1225, "snow" },
    { 1237, "hail" },
    { 1249, "sleet" },
    { 1252, "sleet" },
    { 1255, "snow" },
    { 1258, "snow" },
    { 1261, "hail" },
    { 1264, "hail" },
    { 1087, "thunderstorms-day" },
    { 1273, "thunderstorms-day-rain" },
    { 1276, "thunderstorms-rain" },
    { 1279, "thunderstorms-day-snow" },
    { 1282, "thunderstorms-snow" }
};
```

Konsep:

- Menyimpan pasangan key-value (kode cuaca → nama icon)
- Memudahkan pencarian icon berdasarkan kode

2. Fungsi LoadWeatherIcon

```
public static Image LoadWeatherIcon(int weatherCode, bool isDay)
 {
     try
     {
         string iconName = GetIconName(weatherCode, isDay);
         string iconPath = Path.Combine(iconFolder, $"{iconName}.png");

         if (File.Exists(iconPath))
         {
                  using (var originalImage = Image.FromFile(iconPath))
             {
                 return new Bitmap(originalImage);
             }
         }
         else
         {
             System.Diagnostics.Debug.WriteLine($"Icon not found: {iconPath}");
```

```
                return null!;
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"Error loading icon: {ex.Message}");
            return null!;
        }
    }
```

Proses:

1. Dapatkan nama icon dari kode cuaca
2. Gabungkan path folder dengan nama file
3. Cek apakah file ada
4. Load image dan return sebagai Bitmap

3. Fungsi GetIconName (Private)

```
            string iconName;

            if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
            {
                System.Diagnostics.Debug.WriteLine($"⚠ Weather code
{weatherCode} NOT FOUND in mapping! Using default.");
                iconName = isDay ? "clear-day" : "clear-night";
            }
            else
            {
                System.Diagnostics.Debug.WriteLine($"✓ Weather code {weatherCode}
mapped to: {iconName}");
            }

            if (!isDay)
            {
                if (weatherCode == 1000)
                {
                    iconName = "clear-night";
                }
                else if (weatherCode == 1003)
                {
                    iconName = "partly-cloudy-night";
                }
                else if (weatherCode == 1087)
                {
                    iconName = "thunderstorms-night";
                }
                else if (weatherCode == 1273)
                {
                    iconName = "thunderstorms-night-rain";
                }
                else if (weatherCode == 1279)
                {
                    iconName = "thunderstorms-night-snow";
                }
                else if (iconName.Contains("-day"))
                {"
```

```
                iconName = iconName.Replace("-day", "-night");
            }
        }

        return iconName;
    }
```

Logika:

- Cari nama icon dari dictionary
- Jika tidak ketemu, gunakan default (clear-day/clear-night)
- Jika waktu malam, ganti icon siang dengan malam
- Return nama icon yang sudah disesuaikan

C. Class Root.cs - Data Models

1. Konsep Required Property

```
public class Current
{
    public required string last_updated { get; set; }
    public required object temp_c { get; set; }
    Condition condition { get; set; }
}
```

Penjelasan:

- required - Property wajib diisi saat membuat object
- Mencegah null reference exception
- Compiler akan warning jika property tidak diisi

2. JSON Property Mapping

```
[JsonProperty("us-epa-index")]
public required int usepaindex { get; set; }
```

Fungsi:

- Memetakan property C# ke key JSON yang berbeda
- JSON key: "us-epa-index" (ada strip)
- C# property: usepaindex (tanpa strip)

3. Nested Objects

```
  public class Root
  {
      public required Location location { get; set; }
      public required Current current { get; set; }
      public required Forecast forecast { get; set; }
  }
```

Struktur:

- Root sebagai container utama
- Berisi object Location, Current, Forecast

- Mencerminkan struktur JSON dari API

FITUR-FITUR KHUSUS

1. Icon Dinamis Siang/Malam

Program secara otomatis mengganti icon berdasarkan waktu:

- **is_day = 1** (Siang) → Gunakan icon "-day"
- **is_day = 0** (Malam) → Gunakan icon "-night"

```
LocalIcon(data.current.condition.code,       data.current.is_day       ==       1,
pictureBoxWeather);
```

2. Icon Alert Kondisional

Icon berubah berdasarkan nilai parameter:

Tekanan Udara:

```
if (data.current.pressure_mb >= 1013.25)
 {
     picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-high.png"));
 }
 else
 {
     picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-low.png"));
 }
```

Angin:

```
if (data.current.wind_kph >= 21)
 {
     picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-
alert.png"));
 }
 else
 {
     picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"wind.png"));
 }
```

Curah Hujan:

```
if (data.current.precip_mm >= 11)
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrops.png"));
}
else
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrop.png"));
}
```

3. Indeks UV Dinamis

```
int uvIndex = (int)Math.Floor(data.current.uv);
uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11

string iconName = uvIndex >= 1
    ? $"uv-index-{uvIndex}.png"
    : "not-available.png";

picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));
```

4. Kualitas Udara dengan Deskripsi

```
private string GetAQIDescription(int index)
{
    return index switch
    {
        1 => "Good",
        2 => "Moderate",
        3 => "Unhealthy for Sensitive Groups",
        4 => "Unhealthy",
        5 => "Very Unhealthy",
        6 => "Hazardous",
        _ => "Not Available"
    };
}
```

5. Format Tanggal Forecast

```
DateTime date = DateTime.Parse(forecastDay.date);
lblDate.Text = date.ToString("ddd, MMM dd", CultureInfo.InvariantCulture);
```

Output: **Mon, Jan 15**

6. Enter Key untuk Search

```
private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
{
    if (e.KeyChar == (char)Keys.Enter)
    {
        btnSearch_Click(sender, e);
        e.Handled = true;
    }
}
```

User bisa tekan Enter di TextBox untuk langsung search.

7. Resource Management

```
protected override void OnFormClosing(FormClosingEventArgs e)
 {
     httpClient?.Dispose();
     base.OnFormClosing(e);
 }
```

Dispose HttpClient saat form ditutup untuk membebaskan memory.

LAMPIRAN

Kode Program Lengkap

1. Form1.cs

```
            picDay3 = CreateForecastIcon(panelDay3);
        }

        // Fungsi helper untuk membuat PictureBox icon forecast di dalam panel
        private PictureBox CreateForecastIcon(Panel panel)
        {
            var pictureBox = new PictureBox();
            pictureBox.Location = new Point(53, 23); // Posisi di dalam panel
            pictureBox.Size = new Size(80, 80); // Ukuran 76x71 pixel
            pictureBox.SizeMode = PictureBoxSizeMode.Zoom; // Mode zoom
            pictureBox.BackColor = Color.Transparent; // Background transparan
            panel.Controls.Add(pictureBox); // Tambahkan ke panel
            panel.BringToFront(); // Bawa panel ke depan
            return pictureBox;
        }

        // Fungsi ketika tombol Search diklik
        private async void btnSearch_Click(object sender, EventArgs e)
        {
            // Ambil teks dari textbox dan hapus spasi di awal/akhir
            string city = txtCity.Text.Trim();

            // Validasi: cek apakah nama kota kosong
            if (string.IsNullOrEmpty(city))
            {
                MessageBox.Show("Please enter a city name!", "Warning",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            // Disable tombol search agar tidak bisa diklik saat loading
            btnSearch.Enabled = false;
            btnSearch.Text = "Loading..."; // Ubah teks tombol menjadi "Loading..."
            lblStatus.Text = "Fetching weather data..."; // Update status
            lblStatus.ForeColor = Color.White; // Warna teks putih

            // Panggil Fungsi async untuk mendapatkan data cuaca
            await GetWeatherData(city);

            // Enable kembali tombol search setelah selesai
            btnSearch.Enabled = true;
            btnSearch.Text = "Search"; // Kembalikan teks tombol
        }

        // Fungsi async untuk mengambil data cuaca dari API
        private async Task GetWeatherData(string city)
        {
            try
            {
                // Buat URL dengan parameter: API key, nama kota, jumlah hari forecast, dan AQI
                string url = $"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
                // Kirim request GET ke API
                HttpResponseMessage response = await httpClient.GetAsync(url);

                // Cek apakah response berhasil (status code 200)
                if (response.IsSuccessStatusCode)
                {
                    // Baca content response sebagai string JSON
                    string json = await response.Content.ReadAsStringAsync();
                    // Deserialize JSON menjadi object Root
                    Root weatherData = JsonConvert.DeserializeObject<Root>(json)!;
                    // Tampilkan data cuaca di UI
                    DisplayWeatherData(weatherData);
```

```
                DisplayWeatherData(weatherData);
            }
            else
            {
                // Jika gagal, tampilkan pesan kota tidak ditemukan
                lblStatus.Text = "City not found!";
                lblStatus.ForeColor = Color.FromArgb(255, 100, 100); // Warna merah
                // Bersihkan tampilan cuaca
                ClearWeatherDisplay();
            }
        }
        catch (Exception ex)
        {
            // Tangkap error dan tampilkan pesan error
            MessageBox.Show($"Error: {ex.Message}", "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            lblStatus.Text = "Error loading data";
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
        }
    }

    // Fungsi untuk menampilkan data cuaca ke UI
    private void DisplayWeatherData(Root data)
    {
        // Validasi: cek apakah data null
        if (data == null) return;

        // Tampilkan data lokasi
        lblCity.Text = data.location.name; // Nama kota
        lblCountry.Text = $"{data.location.region}, {data.location.country}"; // Region dan negara
        lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}"; // Waktu update terakhir

        // Tampilkan data cuaca saat ini
        lblTemp.Text = $"{data.current.temp_c}°C"; // Suhu dalam Celsius
        lblCondition.Text = data.current.condition.text; // Kondisi cuaca (text)
        lblFeelsLike.Text = $"Feels like: {data.current.feelslike_c}°C"; // Suhu yang dirasakan

        // Tampilkan data detail cuaca
        lblUVValue.Text = data.current.uv.ToString(); // Indeks UV
        lblHumidityValue.Text = $"{data.current.humidity}%"; // Kelembaban
        lblPressureValue.Text = $"{data.current.pressure_mb} mb"; // Tekanan udara
        lblCloudValue.Text = $"{data.current.cloud}%"; // Persentase awan
        lblWindValue.Text = $"{data.current.wind_kph} km/h\n{data.current.wind_dir}"; // Kecepatan dan arah angin
        lblPrecipValue.Text = $"{data.current.precip_mm} mm"; // Curah hujan

        // Tampilkan data kualitas udara jika tersedia
        if (data.current.air_quality != null)
        {
            // Tampilkan indeks kualitas udara dengan deskripsi
            lblAQValue.Text = $"Air Quality \n{GetAQIDescription(data.current.air_quality.usepaindex)}";
            // Tampilkan PM 2.5 (partikel halus)
            lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1} µg/m³";
            // Tampilkan PM 10 (partikel kasar)
            lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1} µg/m³";
            // Tampilkan Carbon Monoxide
            lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
            // Tampilkan Ozone
            lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
        }

        // Load icon cuaca utama berdasarkan kode cuaca dan waktu (siang/malam)
        LocalIcon(data.current.condition.code, data.current.is_day == 1, pictureBoxWeather);

```

```
            // Tampilkan forecast 3 hari jika data tersedia
            if (data.forecast.forecastday != null && data.forecast.forecastday.Count >= 3)
            {
                DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date, lblDay1Temp, lblDay1Condition, picDay1);
                DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date, lblDay2Temp, lblDay2Condition, picDay2);
                DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date, lblDay3Temp, lblDay3Condition, picDay3);
            }

            // Load icon kelembaban
            picHumidity.LoadAsync(Path.Combine(Application.StartupPath, "icons", "humidity.png"));

            // Load icon tekanan udara (tinggi/rendah berdasarkan nilai tekanan standar 1013.25 mb)
            if (data.current.pressure_mb >= 1013.25)
            {
                picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons", "pressure-high.png"));
            }
            else
            {
                picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons", "pressure-low.png"));
            }

            // Load icon angin (alert jika kecepatan >= 21 km/h)
            if (data.current.wind_kph >= 21)
            {
                picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-alert.png"));
            }
            else
            {
                picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind.png"));
            }

            // Load icon curah hujan (banyak tetes jika >= 11 mm)
            if (data.current.precip_mm >= 11)
            {
                picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons", "raindrops.png"));
            }
            else
            {
                picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons", "raindrop.png"));
            }

            // Load icon UV berdasarkan indeks UV (0-11+)
            int uvIndex = (int)Math.Floor(data.current.uv);
            uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11

            string iconName = uvIndex >= 1
                ? $"uv-index-{uvIndex}.png"
                : "not-available.png";

            picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));

            // Load icon awan berdasarkan persentase awan
            if (data.current.cloud <= 50)
            {
                picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons", "mist.png"));
            }
            else
            {
                picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons", "overcast.png"));
            }

            // Update status berhasil dengan warna hijau
            lblStatus.Text = "Weather data loaded successfully!";
```

```
            lblStatus.ForeColor = Color.FromArgb(150, 255, 150);
        }

        // Fungsi untuk menampilkan data forecast per hari
        private void DisplayForecastDay(Forecastday forecastDay, Label lblDate, Label lblTemp, Label lblCondition, PictureBox pictureBox)
        {
            // Validasi: cek apakah data forecast null
            if (forecastDay == null) return;

            // Parse tanggal dan format menjadi "Day, Month DD" (contoh: Mon, Jan 15)
            DateTime date = DateTime.Parse(forecastDay.date);
            lblDate.Text = date.ToString("ddd, MMM dd", CultureInfo.InvariantCulture);

            // Tampilkan suhu maksimal dan minimal (format: XX°/YY°)
            lblTemp.Text = $"{forecastDay.day.maxtemp_c:F0}°/{forecastDay.day.mintemp_c:F0}°";
            // Tampilkan kondisi cuaca
            lblCondition.Text = forecastDay.day.condition.text;

            // Load icon cuaca dari folder lokal (selalu gunakan icon siang untuk forecast)
            LocalIcon(forecastDay.day.condition.code, true, pictureBox);
        }

        // Fungsi untuk memuat icon cuaca dari folder lokal
        private void LocalIcon(int weatherCode, bool isDay, PictureBox pictureBox)
        {
            try
            {
                // Load icon menggunakan class WeatherIcon
                Image icon = WeatherIcon.LoadWeatherIcon(weatherCode, isDay);
                // Jika icon berhasil dimuat, set ke PictureBox
                if (icon != null)
                    pictureBox.Image = icon;
                else
                    pictureBox.Image = null; // Set null jika gagal
            }
            catch
            {
                // Jika terjadi error, set image null
                pictureBox.Image = null;
            }
        }

        // Fungsi untuk mendapatkan deskripsi kualitas udara berdasarkan indeks
        private string GetAQIDescription(int index)
        {
            return index switch
            {
                1 => "Good", // Baik
                2 => "Moderate", // Sedang
                3 => "Unhealthy for Sensitive Groups", // Tidak sehat untuk kelompok sensitif
                4 => "Unhealthy", // Tidak sehat
                5 => "Very Unhealthy", // Sangat tidak sehat
                6 => "Hazardous", // Berbahaya
                _ => "Not Available" // Tidak tersedia
            };
        }

        // Fungsi untuk membersihkan/mereset tampilan cuaca
        private void ClearWeatherDisplay()
        {
            // Reset semua label menjadi "-"
            lblCity.Text = "-";
```

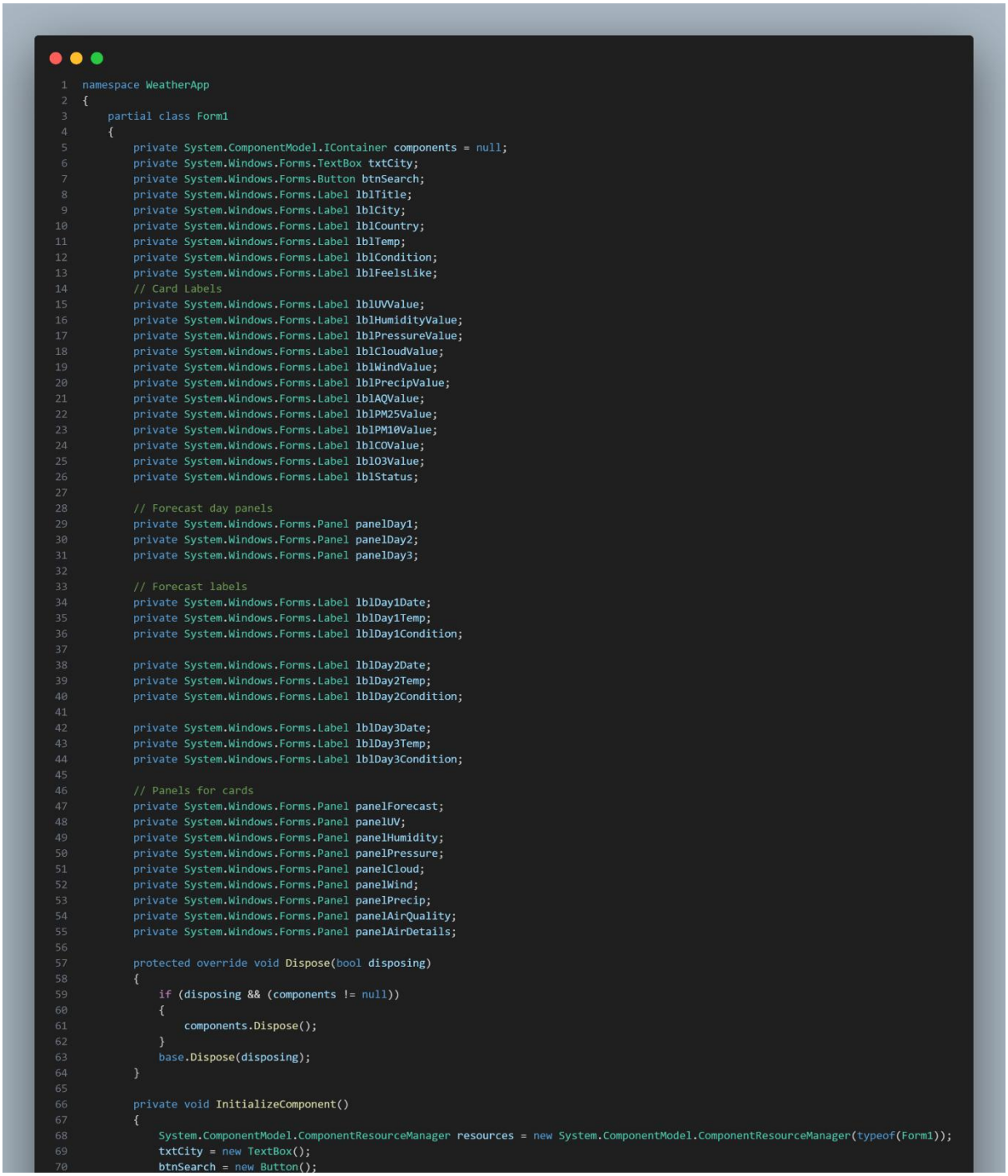
```
            lblCountry.Text = "-";
            lblTemp.Text = "-";
            lblCondition.Text = "-";
            lblFeelsLike.Text = "Feels like: -";
            lblUVValue.Text = "-";
            lblHumidityValue.Text = "-";
            lblPressureValue.Text = "-";
            lblCloudValue.Text = "-";
            lblWindValue.Text = "-";
            lblPrecipValue.Text = "-";
            lblAQValue.Text = "-";
            lblPM25Value.Text = "PM 2.5: -";
            lblPM10Value.Text = "PM10: -";
            lblCOValue.Text = "CO: -";
            lblO3Value.Text = "O3: -";

            // Hapus semua gambar icon
            pictureBoxWeather.Image = null;
            picDay1.Image = null;
            picDay2.Image = null;
            picDay3.Image = null;

            // Reset label forecast 5 hari menjadi "N/A"
            lblDay1Date.Text = "N/A";
            lblDay1Temp.Text = "N/A";
            lblDay1Condition.Text = "N/A";

            lblDay2Date.Text = "N/A";
            lblDay2Temp.Text = "N/A";
            lblDay2Condition.Text = "N/A";

            lblDay3Date.Text = "N/A";
            lblDay3Temp.Text = "N/A";
            lblDay3Condition.Text = "N/A";

        }

        // Fungsi ketika user menekan tombol di textbox
        private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
        {
            // Cek jika tombol yang ditekan adalah Enter
            if (e.KeyChar == (char)Keys.Enter)
            {
                // Trigger event btnSearch_Click (sama seperti klik tombol search)
                btnSearch_Click(sender, e);
                // Set Handled = true agar tidak ada "beep" sound
                e.Handled = true;
            }
        }

        // Override Fungsi OnFormClosing untuk cleanup saat form ditutup
        protected override void OnFormClosing(FormClosingEventArgs e)
        {
            // Dispose HttpClient untuk membebaskan resource
            httpClient?.Dispose();
            // Panggil base Fungsi
            base.OnFormClosing(e);
        }
    }
}
```

2. Root.cs

```cs
using Newtonsoft.Json;
using System.Drawing.Imaging;
using WeatherApp;

namespace WeatherApp
{
    // Class untuk merepresentasikan data kualitas udara
    public class AirQuality
    {
        public required object co { get; set; } // Carbon Monoxide (Karbon Monoksida)
        public required object no2 { get; set; } // Nitrogen Dioxide (Nitrogen Dioksida)
        public required object o3 { get; set; } // Ozone (Ozon)
        public required object so2 { get; set; } // Sulfur Dioxide (Sulfur Dioksida)
        public required object pm2_5 { get; set; } // Particulate Matter 2.5 (Partikel halus 2.5 mikron)
        public required object pm10 { get; set; } // Particulate Matter 10 (Partikel 10 mikron)

        // Indeks kualitas udara US EPA dengan mapping JSON property
        [JsonProperty("us-epa-index")]
        public required int usepaindex { get; set; }

        // Indeks kualitas udara UK DEFRA dengan mapping JSON property
        [JsonProperty("gb-defra-index")]
        public required int gbdefraindex { get; set; }
    }

    // Class untuk merepresentasikan kondisi cuaca
    public class Condition
    {
        public required string text { get; set; } // Deskripsi kondisi cuaca dalam teks (contoh: "Sunny", "Rainy")
        public required string icon { get; set; } // URL icon cuaca dari API
        public required int code { get; set; } // Kode numerik kondisi cuaca (untuk mapping icon lokal)
    }

    // Class untuk merepresentasikan data cuaca saat ini (current weather)
    public class Current
    {
        public required object last_updated_epoch { get; set; } // Waktu update terakhir dalam format epoch/unix timestamp
        public required string last_updated { get; set; } // Waktu update terakhir dalam format string readable
        public required object temp_c { get; set; } // Suhu dalam Celsius
        public required object temp_f { get; set; } // Suhu dalam Fahrenheit
        public required int is_day { get; set; } // Indikator siang/malam (1 = siang, 0 = malam)
        public required Condition condition { get; set; } // Object kondisi cuaca
        public required object wind_mph { get; set; } // Kecepatan angin dalam miles per hour
        public required double wind_kph { get; set; } // Kecepatan angin dalam kilometer per hour
        public required object wind_degree { get; set; } // Arah angin dalam derajat
        public required string wind_dir { get; set; } // Arah angin dalam teks (contoh: "N", "SE", "SW")
        public required double pressure_mb { get; set; } // Tekanan udara dalam millibar
        public required double pressure_in { get; set; } // Tekanan udara dalam inches
        public required double precip_mm { get; set; } // Curah hujan dalam millimeter
        public required double precip_in { get; set; } // Curah hujan dalam inches
        public required double humidity { get; set; } // Kelembaban udara dalam persen
        public required double cloud { get; set; } // Persentase tutupan awan
        public required double feelslike_c { get; set; } // Suhu yang dirasakan dalam Celsius
        public required double feelslike_f { get; set; } // Suhu yang dirasakan dalam Fahrenheit
        public required double vis_km { get; set; } // Jarak pandang/visibility dalam kilometer
        public required double vis_miles { get; set; } // Jarak pandang/visibility dalam miles
        public required double uv { get; set; } // Indeks UV (ultraviolet)
        public required double gust_mph { get; set; } // Kecepatan hembusan angin maksimal dalam miles per hour
        public required object gust_kph { get; set; } // Kecepatan hembusan angin maksimal dalam kilometer per hour
        public required AirQuality air_quality { get; set; } // Object data kualitas udara
```

```
    }

    // Class untuk merepresentasikan data lokasi
    public class Location
    {
        public required string name { get; set; } // Nama kota/lokasi
        public required string region { get; set; } // Nama region/provinsi/state
        public required string country { get; set; } // Nama negara
        public required object lat { get; set; } // Koordinat latitude (garis lintang)
        public required object lon { get; set; } // Koordinat longitude (garis bujur)
        public required string tz_id { get; set; } // Timezone ID (contoh: "Asia/Jakarta")
        public required object localtime_epoch { get; set; } // Waktu lokal dalam format epoch
        public required string localtime { get; set; } // Waktu lokal dalam format string
    }

    // Class untuk merepresentasikan data forecast/ramalan cuaca
    public class Forecast
    {
        public required List<Forecastday> forecastday { get; set; } // List/daftar ramalan cuaca per hari
    }

    // Class untuk merepresentasikan data ramalan cuaca per hari
    public class Forecastday
    {
        public required string date { get; set; } // Tanggal ramalan dalam format string (YYYY-MM-DD)
        public required int date_epoch { get; set; } // Tanggal ramalan dalam format epoch
        public required Day day { get; set; } // Object data cuaca untuk hari tersebut
    }

    // Class untuk merepresentasikan detail cuaca harian
    public class Day
    {
        public required double maxtemp_c { get; set; } // Suhu maksimal dalam Celsius
        public required double maxtemp_f { get; set; } // Suhu maksimal dalam Fahrenheit
        public required double mintemp_c { get; set; } // Suhu minimal dalam Celsius
        public required double mintemp_f { get; set; } // Suhu minimal dalam Fahrenheit
        public required double avgtemp_c { get; set; } // Suhu rata-rata dalam Celsius
        public required double avgtemp_f { get; set; } // Suhu rata-rata dalam Fahrenheit
        public required double maxwind_mph { get; set; } // Kecepatan angin maksimal dalam miles per hour
        public required double maxwind_kph { get; set; } // Kecepatan angin maksimal dalam kilometer per hour
        public required double totalprecip_mm { get; set; } // Total curah hujan dalam millimeter
        public required double totalprecip_in { get; set; } // Total curah hujan dalam inches
        public required double avgvis_km { get; set; } // Rata-rata jarak pandang dalam kilometer
        public required double avgvis_miles { get; set; } // Rata-rata jarak pandang dalam miles
        public required int avghumidity { get; set; } // Rata-rata kelembaban dalam persen
        public required int daily_will_it_rain { get; set; } // Indikator apakah akan hujan (1 = ya, 0 = tidak)
        public required int daily_chance_of_rain { get; set; } // Persentase kemungkinan hujan
        public required int daily_will_it_snow { get; set; } // Indikator apakah akan turun salju (1 = ya, 0 = tidak)
        public required int daily_chance_of_snow { get; set; } // Persentase kemungkinan salju
        public required Condition condition { get; set; } // Object kondisi cuaca
        public required double uv { get; set; } // Indeks UV
    }

    // Class Root sebagai container utama untuk semua data response dari API
    public class Root
    {
        public required Location location { get; set; } // Data lokasi
        public required Current current { get; set; } // Data cuaca saat ini
        public required Forecast forecast { get; set; } // Data ramalan cuaca
        public required object alert { get; set; } // Data alert/peringatan cuaca (jika ada)
    }
}
```

3. Form1.Designer.cs



```
namespace WeatherApp
{
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;
        private System.Windows.Forms.TextBox txtCity;
        private System.Windows.Forms.Button btnSearch;
        private System.Windows.Forms.Label lblTitle;
        private System.Windows.Forms.Label lblCity;
        private System.Windows.Forms.Label lblCountry;
        private System.Windows.Forms.Label lblTemp;
        private System.Windows.Forms.Label lblCondition;
        private System.Windows.Forms.Label lblFeelsLike;
        // Card Labels
        private System.Windows.Forms.Label lblUVValue;
        private System.Windows.Forms.Label lblHumidityValue;
        private System.Windows.Forms.Label lblPressureValue;
        private System.Windows.Forms.Label lblCloudValue;
        private System.Windows.Forms.Label lblWindValue;
        private System.Windows.Forms.Label lblPrecipValue;
        private System.Windows.Forms.Label lblAQValue;
        private System.Windows.Forms.Label lblPM25Value;
        private System.Windows.Forms.Label lblPM10Value;
        private System.Windows.Forms.Label lblCOValue;
        private System.Windows.Forms.Label lblO3Value;
        private System.Windows.Forms.Label lblStatus;

        // Forecast day panels
        private System.Windows.Forms.Panel panelDay1;
        private System.Windows.Forms.Panel panelDay2;
        private System.Windows.Forms.Panel panelDay3;

        // Forecast labels
        private System.Windows.Forms.Label lblDay1Date;
        private System.Windows.Forms.Label lblDay1Temp;
        private System.Windows.Forms.Label lblDay1Condition;

        private System.Windows.Forms.Label lblDay2Date;
        private System.Windows.Forms.Label lblDay2Temp;
        private System.Windows.Forms.Label lblDay2Condition;

        private System.Windows.Forms.Label lblDay3Date;
        private System.Windows.Forms.Label lblDay3Temp;
        private System.Windows.Forms.Label lblDay3Condition;

        // Panels for cards
        private System.Windows.Forms.Panel panelForecast;
        private System.Windows.Forms.Panel panelUV;
        private System.Windows.Forms.Panel panelHumidity;
        private System.Windows.Forms.Panel panelPressure;
        private System.Windows.Forms.Panel panelCloud;
        private System.Windows.Forms.Panel panelWind;
        private System.Windows.Forms.Panel panelPrecip;
        private System.Windows.Forms.Panel panelAirQuality;
        private System.Windows.Forms.Panel panelAirDetails;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            System.ComponentModel.ComponentResourceManager resources = new System.ComponentModel.ComponentResourceManager(typeof(Form1));
            txtCity = new TextBox();
            btnSearch = new Button();
```

```
            lblTitle = new Label();
            lblCity = new Label();
            lblCountry = new Label();
            lblTemp = new Label();
            lblCondition = new Label();
            lblFeelsLike = new Label();
            lblUVValue = new Label();
            lblHumidityValue = new Label();
            lblPressureValue = new Label();
            lblCloudValue = new Label();
            lblWindValue = new Label();
            lblPrecipValue = new Label();
            lblAQValue = new Label();
            lblPM25Value = new Label();
            lblPM10Value = new Label();
            lblCOValue = new Label();
            lblO3Value = new Label();
            lblStatus = new Label();
            panelDay1 = new Panel();
            lblDay1Date = new Label();
            lblDay1Temp = new Label();
            lblDay1Condition = new Label();
            panelDay2 = new Panel();
            lblDay2Date = new Label();
            lblDay2Temp = new Label();
            lblDay2Condition = new Label();
            panelDay3 = new Panel();
            lblDay3Date = new Label();
            lblDay3Temp = new Label();
            lblDay3Condition = new Label();
            panelForecast = new Panel();
            lblLastUpdate = new Label();
            panelUV = new Panel();
            picUV = new PictureBox();
            lblTitleUV = new Label();
            panelHumidity = new Panel();
            picHumidity = new PictureBox();
            lblTitleHumidity = new Label();
            panelPressure = new Panel();
            picPressure = new PictureBox();
            lblTitlePressure = new Label();
            panelCloud = new Panel();
            picCloud = new PictureBox();
            lblTitleCloud = new Label();
            panelWind = new Panel();
            picWind = new PictureBox();
            lblTitleWind = new Label();
            panelPrecip = new Panel();
            picPrecip = new PictureBox();
            lblTitlePrecip = new Label();
            panelAirQuality = new Panel();
            panelAirDetails = new Panel();
            panelDay1.SuspendLayout();
            panelDay2.SuspendLayout();
            panelDay3.SuspendLayout();
            panelForecast.SuspendLayout();
            panelUV.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picUV).BeginInit();
            panelHumidity.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picHumidity).BeginInit();
            panelPressure.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picPressure).BeginInit();
            panelCloud.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picCloud).BeginInit();
            panelWind.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picWind).BeginInit();
            panelPrecip.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)picPrecip).BeginInit();
            panelAirQuality.SuspendLayout();
            panelAirDetails.SuspendLayout();
            SuspendLayout();
            //
            // txtCity
            //
```

```
            txtCity.Font = new Font("Segoe UI", 12F);
            txtCity.Location = new Point(29, 60);
            txtCity.Margin = new Padding(3, 4, 3, 4);
            txtCity.Name = "txtCity";
            txtCity.Size = new Size(491, 34);
            txtCity.TabIndex = 1;
            txtCity.KeyPress += txtCity_KeyPress;
            //
            // btnSearch
            //
            btnSearch.BackColor = Color.Transparent;
            btnSearch.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            btnSearch.BackgroundImageLayout = ImageLayout.Stretch;
            btnSearch.Cursor = Cursors.Hand;
            btnSearch.FlatAppearance.BorderSize = 0;
            btnSearch.FlatStyle = FlatStyle.Flat;
            btnSearch.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
            btnSearch.ForeColor = Color.White;
            btnSearch.Location = new Point(526, 55);
            btnSearch.Margin = new Padding(3, 4, 3, 4);
            btnSearch.Name = "btnSearch";
            btnSearch.Size = new Size(137, 39);
            btnSearch.TabIndex = 2;
            btnSearch.Text = "Search";
            btnSearch.UseVisualStyleBackColor = false;
            btnSearch.Click += btnSearch_Click;
            //
            // lblTitle
            //
            lblTitle.AutoSize = true;
            lblTitle.BackColor = Color.Transparent;
            lblTitle.Font = new Font("Segoe UI", 18F, FontStyle.Bold);
            lblTitle.ForeColor = Color.White;
            lblTitle.Location = new Point(29, 15);
            lblTitle.Name = "lblTitle";
            lblTitle.Size = new Size(160, 41);
            lblTitle.TabIndex = 0;
            lblTitle.Text = "WEATHER";
            //
            // lblCity
            //
            lblCity.AutoSize = true;
            lblCity.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
            lblCity.ForeColor = Color.White;
            lblCity.Location = new Point(197, 47);
            lblCity.Name = "lblCity";
            lblCity.Size = new Size(34, 46);
            lblCity.TabIndex = 1;
            lblCity.Text = "-";
            //
            // lblCountry
            //
            lblCountry.AutoSize = true;
            lblCountry.Font = new Font("Segoe UI", 11F);
            lblCountry.ForeColor = Color.FromArgb(220, 230, 240);
            lblCountry.Location = new Point(197, 93);
            lblCountry.Name = "lblCountry";
            lblCountry.Size = new Size(20, 25);
            lblCountry.TabIndex = 2;
            lblCountry.Text = "-";
            //
            // lblTemp
            //
            lblTemp.AutoSize = true;
            lblTemp.Font = new Font("Segoe UI", 48F, FontStyle.Bold);
            lblTemp.ForeColor = Color.White;
            lblTemp.Location = new Point(183, 127);
            lblTemp.Name = "lblTemp";
            lblTemp.Size = new Size(77, 106);
            lblTemp.TabIndex = 3;
            lblTemp.Text = "-";
            //
            // lblCondition
            //
            lblCondition.AutoSize = true;
```

```
            lblCondition.Font = new Font("Segoe UI", 10.8F, FontStyle.Regular, GraphicsUnit.Point, 0);
            lblCondition.ForeColor = Color.FromArgb(220, 230, 240);
            lblCondition.Location = new Point(447, 188);
            lblCondition.Name = "lblCondition";
            lblCondition.Size = new Size(19, 25);
            lblCondition.TabIndex = 4;
            lblCondition.Text = "-";
            //
            // lblFeelsLike
            //
            lblFeelsLike.AutoSize = true;
            lblFeelsLike.Font = new Font("Segoe UI", 10F);
            lblFeelsLike.ForeColor = Color.FromArgb(200, 220, 230);
            lblFeelsLike.Location = new Point(197, 245);
            lblFeelsLike.Name = "lblFeelsLike";
            lblFeelsLike.Size = new Size(93, 23);
            lblFeelsLike.TabIndex = 5;
            lblFeelsLike.Text = "Feels like: -";
            //
            // lblUVValue
            //
            lblUVValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
            lblUVValue.ForeColor = Color.White;
            lblUVValue.Location = new Point(0, 55);
            lblUVValue.Name = "lblUVValue";
            lblUVValue.Size = new Size(169, 67);
            lblUVValue.TabIndex = 1;
            lblUVValue.Text = "-";
            lblUVValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblHumidityValue
            //
            lblHumidityValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
            lblHumidityValue.ForeColor = Color.White;
            lblHumidityValue.Location = new Point(0, 55);
            lblHumidityValue.Name = "lblHumidityValue";
            lblHumidityValue.Size = new Size(180, 67);
            lblHumidityValue.TabIndex = 1;
            lblHumidityValue.Text = "-";
            lblHumidityValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblPressureValue
            //
            lblPressureValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
            lblPressureValue.ForeColor = Color.White;
            lblPressureValue.Location = new Point(0, 67);
            lblPressureValue.Name = "lblPressureValue";
            lblPressureValue.Size = new Size(169, 67);
            lblPressureValue.TabIndex = 1;
            lblPressureValue.Text = "-";
            lblPressureValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblCloudValue
            //
            lblCloudValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
            lblCloudValue.ForeColor = Color.White;
            lblCloudValue.Location = new Point(0, 67);
            lblCloudValue.Name = "lblCloudValue";
            lblCloudValue.Size = new Size(180, 67);
            lblCloudValue.TabIndex = 1;
            lblCloudValue.Text = "-";
            lblCloudValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblWindValue
            //
            lblWindValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
            lblWindValue.ForeColor = Color.White;
            lblWindValue.Location = new Point(0, 55);
            lblWindValue.Name = "lblWindValue";
            lblWindValue.Size = new Size(169, 121);
            lblWindValue.TabIndex = 1;
            lblWindValue.Text = "-";
            //
            // lblPrecipValue
            //
```

```
            //
            lblPrecipValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
            lblPrecipValue.ForeColor = Color.White;
            lblPrecipValue.Location = new Point(0, 55);
            lblPrecipValue.Name = "lblPrecipValue";
            lblPrecipValue.Size = new Size(180, 67);
            lblPrecipValue.TabIndex = 1;
            lblPrecipValue.Text = "-";
            lblPrecipValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblAQValue
            //
            lblAQValue.Font = new Font("Segoe UI", 16F, FontStyle.Bold);
            lblAQValue.ForeColor = Color.White;
            lblAQValue.Location = new Point(23, 24);
            lblAQValue.Name = "lblAQValue";
            lblAQValue.Size = new Size(617, 95);
            lblAQValue.TabIndex = 1;
            lblAQValue.Text = "-";
            lblAQValue.TextAlign = ContentAlignment.TopCenter;
            //
            // lblPM25Value
            //
            lblPM25Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
            lblPM25Value.ForeColor = Color.FromArgb(200, 220, 240);
            lblPM25Value.Location = new Point(23, 16);
            lblPM25Value.Name = "lblPM25Value";
            lblPM25Value.Size = new Size(617, 33);
            lblPM25Value.TabIndex = 0;
            lblPM25Value.Text = "PM 2.5: -";
            lblPM25Value.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblPM10Value
            //
            lblPM10Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
            lblPM10Value.ForeColor = Color.FromArgb(200, 220, 240);
            lblPM10Value.Location = new Point(23, 49);
            lblPM10Value.Name = "lblPM10Value";
            lblPM10Value.Size = new Size(617, 33);
            lblPM10Value.TabIndex = 1;
            lblPM10Value.Text = "PM10: -";
            lblPM10Value.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblCOValue
            //
            lblCOValue.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
            lblCOValue.ForeColor = Color.FromArgb(200, 220, 240);
            lblCOValue.Location = new Point(23, 82);
            lblCOValue.Name = "lblCOValue";
            lblCOValue.Size = new Size(617, 33);
            lblCOValue.TabIndex = 2;
            lblCOValue.Text = "CO: -";
            lblCOValue.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblO3Value
            //
            lblO3Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
            lblO3Value.ForeColor = Color.FromArgb(200, 220, 240);
            lblO3Value.Location = new Point(23, 115);
            lblO3Value.Name = "lblO3Value";
            lblO3Value.Size = new Size(617, 33);
            lblO3Value.TabIndex = 3;
            lblO3Value.Text = "O3: -";
            lblO3Value.TextAlign = ContentAlignment.MiddleLeft;
            //
            // lblStatus
            //
            lblStatus.AutoSize = true;
            lblStatus.Font = new Font("Segoe UI", 9F);
            lblStatus.ForeColor = Color.White;
            lblStatus.Location = new Point(20, 1750);
            lblStatus.Name = "lblStatus";
            lblStatus.Size = new Size(226, 20);
            lblStatus.TabIndex = 18;
            lblStatus.Text = "Enter city name and press Search";
```

```
            //
            // panelDay1
            //
            panelDay1.BackColor = Color.Transparent;
            panelDay1.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelDay1.BackgroundImageLayout = ImageLayout.Stretch;
            panelDay1.Controls.Add(lblDay1Date);
            panelDay1.Controls.Add(lblDay1Temp);
            panelDay1.Controls.Add(lblDay1Condition);
            panelDay1.ForeColor = Color.FromArgb(220, 220, 220);
            panelDay1.Location = new Point(20, 442);
            panelDay1.Margin = new Padding(3, 4, 3, 4);
            panelDay1.Name = "panelDay1";
            panelDay1.Size = new Size(217, 213);
            panelDay1.TabIndex = 5;
            //
            // lblDay1Date
            //
            lblDay1Date.BackColor = Color.Transparent;
            lblDay1Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
            lblDay1Date.ForeColor = Color.White;
            lblDay1Date.Location = new Point(6, 13);
            lblDay1Date.Name = "lblDay1Date";
            lblDay1Date.Size = new Size(208, 27);
            lblDay1Date.TabIndex = 0;
            lblDay1Date.Text = "N/A";
            lblDay1Date.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay1Temp
            //
            lblDay1Temp.BackColor = Color.Transparent;
            lblDay1Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
            lblDay1Temp.ForeColor = Color.White;
            lblDay1Temp.Location = new Point(6, 133);
            lblDay1Temp.Name = "lblDay1Temp";
            lblDay1Temp.Size = new Size(208, 33);
            lblDay1Temp.TabIndex = 1;
            lblDay1Temp.Text = "N/A";
            lblDay1Temp.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay1Condition
            //
            lblDay1Condition.BackColor = Color.Transparent;
            lblDay1Condition.Font = new Font("Segoe UI", 8F);
            lblDay1Condition.ForeColor = Color.FromArgb(220, 230, 240);
            lblDay1Condition.Location = new Point(6, 173);
            lblDay1Condition.Name = "lblDay1Condition";
            lblDay1Condition.Size = new Size(208, 33);
            lblDay1Condition.TabIndex = 2;
            lblDay1Condition.Text = " N/A";
            lblDay1Condition.TextAlign = ContentAlignment.TopCenter;
            //
            // panelDay2
            //
            panelDay2.BackColor = Color.Transparent;
            panelDay2.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelDay2.BackgroundImageLayout = ImageLayout.Stretch;
            panelDay2.Controls.Add(lblDay2Date);
            panelDay2.Controls.Add(lblDay2Temp);
            panelDay2.Controls.Add(lblDay2Condition);
            panelDay2.ForeColor = Color.FromArgb(220, 220, 220);
            panelDay2.Location = new Point(243, 442);
            panelDay2.Margin = new Padding(3, 4, 3, 4);
            panelDay2.Name = "panelDay2";
            panelDay2.Size = new Size(217, 213);
            panelDay2.TabIndex = 6;
            //
            // lblDay2Date
            //
            lblDay2Date.BackColor = Color.Transparent;
            lblDay2Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
            lblDay2Date.ForeColor = Color.White;
            lblDay2Date.Location = new Point(6, 13);
            lblDay2Date.Name = "lblDay2Date";
            lblDay2Date.Size = new Size(208, 27);
```

```
            lblDay2Date.TabIndex = 0;
            lblDay2Date.Text = "N/A";
            lblDay2Date.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay2Temp
            //
            lblDay2Temp.BackColor = Color.Transparent;
            lblDay2Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
            lblDay2Temp.ForeColor = Color.White;
            lblDay2Temp.Location = new Point(6, 133);
            lblDay2Temp.Name = "lblDay2Temp";
            lblDay2Temp.Size = new Size(208, 33);
            lblDay2Temp.TabIndex = 1;
            lblDay2Temp.Text = "N/A";
            lblDay2Temp.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay2Condition
            //
            lblDay2Condition.BackColor = Color.Transparent;
            lblDay2Condition.Font = new Font("Segoe UI", 8F);
            lblDay2Condition.ForeColor = Color.FromArgb(220, 230, 240);
            lblDay2Condition.Location = new Point(6, 173);
            lblDay2Condition.Name = "lblDay2Condition";
            lblDay2Condition.Size = new Size(208, 33);
            lblDay2Condition.TabIndex = 2;
            lblDay2Condition.Text = "N/A";
            lblDay2Condition.TextAlign = ContentAlignment.TopCenter;
            //
            // panelDay3
            //
            panelDay3.BackColor = Color.Transparent;
            panelDay3.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelDay3.BackgroundImageLayout = ImageLayout.Stretch;
            panelDay3.Controls.Add(lblDay3Date);
            panelDay3.Controls.Add(lblDay3Temp);
            panelDay3.Controls.Add(lblDay3Condition);
            panelDay3.ForeColor = Color.FromArgb(220, 220, 220);
            panelDay3.Location = new Point(466, 442);
            panelDay3.Margin = new Padding(3, 4, 3, 4);
            panelDay3.Name = "panelDay3";
            panelDay3.Size = new Size(217, 213);
            panelDay3.TabIndex = 7;
            //
            // lblDay3Date
            //
            lblDay3Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
            lblDay3Date.ForeColor = Color.White;
            lblDay3Date.Location = new Point(6, 13);
            lblDay3Date.Name = "lblDay3Date";
            lblDay3Date.Size = new Size(208, 27);
            lblDay3Date.TabIndex = 0;
            lblDay3Date.Text = "N/A";
            lblDay3Date.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay3Temp
            //
            lblDay3Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
            lblDay3Temp.ForeColor = Color.White;
            lblDay3Temp.Location = new Point(6, 133);
            lblDay3Temp.Name = "lblDay3Temp";
            lblDay3Temp.Size = new Size(208, 33);
            lblDay3Temp.TabIndex = 1;
            lblDay3Temp.Text = "N/A";
            lblDay3Temp.TextAlign = ContentAlignment.MiddleCenter;
            //
            // lblDay3Condition
            //
            lblDay3Condition.Font = new Font("Segoe UI", 8F);
            lblDay3Condition.ForeColor = Color.FromArgb(220, 230, 240);
            lblDay3Condition.Location = new Point(6, 173);
            lblDay3Condition.Name = "lblDay3Condition";
            lblDay3Condition.Size = new Size(208, 33);
            lblDay3Condition.TabIndex = 2;
            lblDay3Condition.Text = "N/A";
```

```
            lblDay3Condition.TextAlign = ContentAlignment.TopCenter;
            //
            // panelForecast
            //
            panelForecast.BackColor = Color.Transparent;
            panelForecast.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelForecast.BackgroundImageLayout = ImageLayout.Stretch;
            panelForecast.Controls.Add(lblLastUpdate);
            panelForecast.Controls.Add(lblCity);
            panelForecast.Controls.Add(lblCountry);
            panelForecast.Controls.Add(lblTemp);
            panelForecast.Controls.Add(lblCondition);
            panelForecast.Controls.Add(lblFeelsLike);
            panelForecast.ForeColor = Color.White;
            panelForecast.Location = new Point(20, 123);
            panelForecast.Margin = new Padding(3, 4, 3, 4);
            panelForecast.Name = "panelForecast";
            panelForecast.Size = new Size(663, 293);
            panelForecast.TabIndex = 4;
            //
            // lblLastUpdate
            //
            lblLastUpdate.AutoSize = true;
            lblLastUpdate.Font = new Font("Segoe UI", 10.2F, FontStyle.Regular, GraphicsUnit.Point, 0);
            lblLastUpdate.ForeColor = Color.FromArgb(220, 230, 240);
            lblLastUpdate.Location = new Point(447, 229);
            lblLastUpdate.Name = "lblLastUpdate";
            lblLastUpdate.Size = new Size(17, 23);
            lblLastUpdate.TabIndex = 6;
            lblLastUpdate.Text = "-";
            //
            // panelUV
            //
            panelUV.BackColor = Color.Transparent;
            panelUV.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelUV.BackgroundImageLayout = ImageLayout.Stretch;
            panelUV.Controls.Add(picUV);
            panelUV.Controls.Add(lblTitleUV);
            panelUV.Controls.Add(lblUVValue);
            panelUV.ForeColor = Color.FromArgb(220, 220, 220);
            panelUV.Location = new Point(20, 688);
            panelUV.Margin = new Padding(3, 4, 3, 4);
            panelUV.Name = "panelUV";
            panelUV.Size = new Size(320, 213);
            panelUV.TabIndex = 10;
            //
            // picUV
            //
            picUV.BackgroundImageLayout = ImageLayout.Zoom;
            picUV.ErrorImage = null;
            picUV.InitialImage = null;
            picUV.Location = new Point(144, 3);
            picUV.Name = "picUV";
            picUV.Size = new Size(173, 207);
            picUV.SizeMode = PictureBoxSizeMode.Zoom;
            picUV.TabIndex = 8;
            picUV.TabStop = false;
            //
            // lblTitleUV
            //
            lblTitleUV.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
            lblTitleUV.ForeColor = Color.Transparent;
            lblTitleUV.Location = new Point(0, 0);
            lblTitleUV.Name = "lblTitleUV";
            lblTitleUV.Size = new Size(61, 55);
            lblTitleUV.TabIndex = 2;
            lblTitleUV.Text = "UV";
            lblTitleUV.TextAlign = ContentAlignment.BottomLeft;
            //
            // panelHumidity
            //
            panelHumidity.BackColor = Color.Transparent;
            panelHumidity.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelHumidity.BackgroundImageLayout = ImageLayout.Stretch;
            panelHumidity.Controls.Add(picHumidity);
```

```
            panelHumidity.Controls.Add(lblTitleHumidity);
            panelHumidity.Controls.Add(lblHumidityValue);
            panelHumidity.ForeColor = Color.FromArgb(220, 220, 220);
            panelHumidity.Location = new Point(363, 688);
            panelHumidity.Margin = new Padding(3, 4, 3, 4);
            panelHumidity.Name = "panelHumidity";
            panelHumidity.Size = new Size(320, 213);
            panelHumidity.TabIndex = 11;
            //
            // picHumidity
            //
            picHumidity.BackColor = Color.Transparent;
            picHumidity.BackgroundImageLayout = ImageLayout.Zoom;
            picHumidity.ErrorImage = null;
            picHumidity.InitialImage = null;
            picHumidity.Location = new Point(148, 3);
            picHumidity.Name = "picHumidity";
            picHumidity.Size = new Size(172, 207);
            picHumidity.SizeMode = PictureBoxSizeMode.Zoom;
            picHumidity.TabIndex = 9;
            picHumidity.TabStop = false;
            //
            // lblTitleHumidity
            //
            lblTitleHumidity.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
            lblTitleHumidity.ForeColor = Color.Transparent;
            lblTitleHumidity.Location = new Point(0, 0);
            lblTitleHumidity.Name = "lblTitleHumidity";
            lblTitleHumidity.Size = new Size(155, 55);
            lblTitleHumidity.TabIndex = 3;
            lblTitleHumidity.Text = "Humidity";
            lblTitleHumidity.TextAlign = ContentAlignment.BottomLeft;
            //
            // panelPressure
            //
            panelPressure.BackColor = Color.Transparent;
            panelPressure.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
            panelPressure.BackgroundImageLayout = ImageLayout.Stretch;
            panelPressure.Controls.Add(picPressure);
            panelPressure.Controls.Add(lblTitlePressure);
            panelPressure.Controls.Add(lblPressureValue);
            panelPressure.ForeColor = Color.FromArgb(220, 220, 220);
            panelPressure.Location = new Point(20, 928);
            panelPressure.Margin = new Padding(3, 4, 3, 4);
            panelPressure.Name = "panelPressure";
            panelPressure.Size = new Size(320, 213);
            panelPressure.TabIndex = 12;
            //
            // picPressure
            //
            picPressure.BackgroundImageLayout = ImageLayout.Stretch;
            picPressure.ErrorImage = null;
            picPressure.InitialImage = null;
            picPressure.Location = new Point(144, 3);
            picPressure.Name = "picPressure";
            picPressure.Size = new Size(173, 207);
            picPressure.SizeMode = PictureBoxSizeMode.Zoom;
            picPressure.TabIndex = 9;
            picPressure.TabStop = false;
            //
            // lblTitlePressure
            //
            lblTitlePressure.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
            lblTitlePressure.ForeColor = Color.Transparent;
            lblTitlePressure.Location = new Point(0, 0);
            lblTitlePressure.Name = "lblTitlePressure";
            lblTitlePressure.Size = new Size(155, 55);
            lblTitlePressure.TabIndex = 4;
            lblTitlePressure.Text = "Pressure";
            lblTitlePressure.TextAlign = ContentAlignment.BottomLeft;
            //
            // panelCloud
            //
            panelCloud.BackColor = Color.Transparent;
```

```
667             panelCloud.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
668             panelCloud.BackgroundImageLayout = ImageLayout.Stretch;
669             panelCloud.Controls.Add(picCloud);
670             panelCloud.Controls.Add(lblTitleCloud);
671             panelCloud.Controls.Add(lblCloudValue);
672             panelCloud.ForeColor = Color.FromArgb(220, 220, 220);
673             panelCloud.Location = new Point(363, 928);
674             panelCloud.Margin = new Padding(3, 4, 3, 4);
675             panelCloud.Name = "panelCloud";
676             panelCloud.Size = new Size(320, 213);
677             panelCloud.TabIndex = 13;
678             //
679             // picCloud
680             //
681             picCloud.BackgroundImageLayout = ImageLayout.Zoom;
682             picCloud.ErrorImage = null;
683             picCloud.InitialImage = null;
684             picCloud.Location = new Point(148, 3);
685             picCloud.Name = "picCloud";
686             picCloud.Size = new Size(169, 207);
687             picCloud.SizeMode = PictureBoxSizeMode.Zoom;
688             picCloud.TabIndex = 9;
689             picCloud.TabStop = false;
690             //
691             // lblTitleCloud
692             //
693             lblTitleCloud.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
694             lblTitleCloud.ForeColor = Color.Transparent;
695             lblTitleCloud.Location = new Point(0, 0);
696             lblTitleCloud.Name = "lblTitleCloud";
697             lblTitleCloud.Size = new Size(155, 55);
698             lblTitleCloud.TabIndex = 4;
699             lblTitleCloud.Text = "Cloud";
700             lblTitleCloud.TextAlign = ContentAlignment.BottomLeft;
701             //
702             // panelWind
703             //
704             panelWind.BackColor = Color.Transparent;
705             panelWind.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
706             panelWind.BackgroundImageLayout = ImageLayout.Stretch;
707             panelWind.Controls.Add(picWind);
708             panelWind.Controls.Add(lblTitleWind);
709             panelWind.Controls.Add(lblWindValue);
710             panelWind.ForeColor = Color.FromArgb(220, 220, 220);
711             panelWind.Location = new Point(20, 1168);
712             panelWind.Margin = new Padding(3, 4, 3, 4);
713             panelWind.Name = "panelWind";
714             panelWind.Size = new Size(320, 213);
715             panelWind.TabIndex = 14;
716             //
717             // picWind
718             //
719             picWind.BackgroundImageLayout = ImageLayout.Zoom;
720             picWind.ErrorImage = null;
721             picWind.InitialImage = null;
722             picWind.Location = new Point(161, 3);
723             picWind.Name = "picWind";
724             picWind.Size = new Size(156, 207);
725             picWind.SizeMode = PictureBoxSizeMode.Zoom;
726             picWind.TabIndex = 9;
727             picWind.TabStop = false;
728             //
729             // lblTitleWind
730             //
731             lblTitleWind.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
732             lblTitleWind.ForeColor = Color.Transparent;
733             lblTitleWind.Location = new Point(0, 0);
734             lblTitleWind.Name = "lblTitleWind";
735             lblTitleWind.Size = new Size(155, 55);
736             lblTitleWind.TabIndex = 4;
737             lblTitleWind.Text = "Wind";
738             lblTitleWind.TextAlign = ContentAlignment.BottomLeft;
739             //
740             // panelPrecip
741             //
```

```
742             panelPrecip.BackColor = Color.Transparent;
743             panelPrecip.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
744             panelPrecip.BackgroundImageLayout = ImageLayout.Stretch;
745             panelPrecip.Controls.Add(picPrecip);
746             panelPrecip.Controls.Add(lblTitlePrecip);
747             panelPrecip.Controls.Add(lblPrecipValue);
748             panelPrecip.ForeColor = Color.FromArgb(220, 220, 220);
749             panelPrecip.Location = new Point(363, 1168);
750             panelPrecip.Margin = new Padding(3, 4, 3, 4);
751             panelPrecip.Name = "panelPrecip";
752             panelPrecip.Size = new Size(320, 213);
753             panelPrecip.TabIndex = 15;
754             //
755             // picPrecip
756             //
757             picPrecip.BackgroundImageLayout = ImageLayout.Zoom;
758             picPrecip.ErrorImage = null;
759             picPrecip.InitialImage = null;
760             picPrecip.Location = new Point(148, 55);
761             picPrecip.Name = "picPrecip";
762             picPrecip.Size = new Size(172, 155);
763             picPrecip.SizeMode = PictureBoxSizeMode.Zoom;
764             picPrecip.TabIndex = 9;
765             picPrecip.TabStop = false;
766             //
767             // lblTitlePrecip
768             //
769             lblTitlePrecip.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
770             lblTitlePrecip.ForeColor = Color.Transparent;
771             lblTitlePrecip.Location = new Point(0, 0);
772             lblTitlePrecip.Name = "lblTitlePrecip";
773             lblTitlePrecip.Size = new Size(204, 55);
774             lblTitlePrecip.TabIndex = 4;
775             lblTitlePrecip.Text = "Precipitation ";
776             lblTitlePrecip.TextAlign = ContentAlignment.BottomLeft;
777             //
778             // panelAirQuality
779             //
780             panelAirQuality.BackColor = Color.Transparent;
781             panelAirQuality.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
782             panelAirQuality.BackgroundImageLayout = ImageLayout.Stretch;
783             panelAirQuality.Controls.Add(lblAQValue);
784             panelAirQuality.ForeColor = Color.FromArgb(220, 220, 220);
785             panelAirQuality.Location = new Point(20, 1404);
786             panelAirQuality.Margin = new Padding(3, 4, 3, 4);
787             panelAirQuality.Name = "panelAirQuality";
788             panelAirQuality.Size = new Size(663, 133);
789             panelAirQuality.TabIndex = 16;
790             //
791             // panelAirDetails
792             //
793             panelAirDetails.BackColor = Color.Transparent;
794             panelAirDetails.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
795             panelAirDetails.BackgroundImageLayout = ImageLayout.Stretch;
796             panelAirDetails.Controls.Add(lblPM25Value);
797             panelAirDetails.Controls.Add(lblPM10Value);
798             panelAirDetails.Controls.Add(lblCOValue);
799             panelAirDetails.Controls.Add(lblO3Value);
800             panelAirDetails.ForeColor = Color.FromArgb(220, 220, 220);
801             panelAirDetails.Location = new Point(20, 1557);
802             panelAirDetails.Margin = new Padding(3, 4, 3, 4);
803             panelAirDetails.Name = "panelAirDetails";
804             panelAirDetails.Size = new Size(663, 173);
805             panelAirDetails.TabIndex = 17;
806             //
807             // Form1
808             //
809             AutoScaleDimensions = new SizeF(8F, 20F);
810             AutoScaleMode = AutoScaleMode.Font;
811             AutoScroll = true;
812             BackColor = Color.FromArgb(52, 108, 160);
813             BackgroundImage = ProjectAkhir.Resources.Resources.bg;
814             BackgroundImageLayout = ImageLayout.Stretch;
815             ClientSize = new Size(718, 1055);
```

```
            Controls.Add(lblStatus);
            Controls.Add(panelAirDetails);
            Controls.Add(panelAirQuality);
            Controls.Add(panelPrecip);
            Controls.Add(panelWind);
            Controls.Add(panelCloud);
            Controls.Add(panelPressure);
            Controls.Add(panelHumidity);
            Controls.Add(panelUV);
            Controls.Add(panelDay3);
            Controls.Add(panelDay2);
            Controls.Add(panelDay1);
            Controls.Add(panelForecast);
            Controls.Add(btnSearch);
            Controls.Add(txtCity);
            Controls.Add(lblTitle);
            Icon = (Icon)resources.GetObject("$this.Icon");
            Margin = new Padding(3, 4, 3, 4);
            Name = "Form1";
            StartPosition = FormStartPosition.CenterScreen;
            Text = "Weather";
            panelDay1.ResumeLayout(false);
            panelDay2.ResumeLayout(false);
            panelDay3.ResumeLayout(false);
            panelForecast.ResumeLayout(false);
            panelForecast.PerformLayout();
            panelUV.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picUV).EndInit();
            panelHumidity.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picHumidity).EndInit();
            panelPressure.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picPressure).EndInit();
            panelCloud.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picCloud).EndInit();
            panelWind.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picWind).EndInit();
            panelPrecip.ResumeLayout(false);
            ((System.ComponentModel.ISupportInitialize)picPrecip).EndInit();
            panelAirQuality.ResumeLayout(false);
            panelAirDetails.ResumeLayout(false);
            ResumeLayout(false);
            PerformLayout();
        }

        private System.Windows.Forms.Label CreateCardLabel(string text, int top, int width)
        {
            var label = new System.Windows.Forms.Label();
            label.Font = new System.Drawing.Font("Segoe UI", 13F, System.Drawing.FontStyle.Bold);
            label.ForeColor = System.Drawing.Color.FromArgb(100, 130, 160);
            label.Location = new System.Drawing.Point((280 - width) / 2, top);
            label.Name = "lbl" + text + "Title";
            label.Size = new System.Drawing.Size(width, 25);
            label.TabIndex = 0;
            label.Text = text.ToUpper();
            label.TextAlign = System.Drawing.ContentAlignment.MiddleCenter;
            return label;
        }
        private Label lblTitleUV;
        private Label lblTitleHumidity;
        private Label lblTitlePressure;
        private Label lblTitleCloud;
        private Label lblTitleWind;
        private Label lblTitlePrecip;
        private Label lblLastUpdate;
        private PictureBox picUV;
        private PictureBox picHumidity;
        private PictureBox picPressure;
        private PictureBox picCloud;
        private PictureBox picWind;
        private PictureBox picPrecip;
    }
}
```

4. WeatherIcon.cs

```cs
using System;
using System.Collections.Generic;
using System.Drawing;
using System.IO;
using System.Windows.Forms;
using WeatherApp;

namespace WeatherApp
{
    public static class WeatherIcon
    {
        // Dictionary untuk menyimpan mapping antara kode cuaca dengan nama icon
        private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int, string>();
        // Variable untuk menyimpan path folder icon
        private static string iconFolder = Path.Combine(Application.StartupPath, "icons");

        // Constructor static yang dipanggil pertama kali saat class digunakan
        static WeatherIcon()
        {
            InitializeWeatherCodes();
        }

        // Fungsi untuk mengatur path folder icon
        public static void IconFolder(string folderPath)
        {
            iconFolder = folderPath;
        }

        // Fungsi untuk menginisialisasi mapping kode cuaca ke nama icon
        private static void InitializeWeatherCodes()
        {
            weatherCodeToIcon = new Dictionary<int, string>
            {
                // Cerah/Berawan
                { 1000, "clear-day" },

                // Cerah Berawan
                { 1003, "partly-cloudy-day" },

                // Berawan
                { 1006, "cloudy" },
                { 1009, "overcast" },

                // Kabut
                { 1030, "mist" },
                { 1135, "fog" },
                { 1147, "fog" },

                // Gerimis
                { 1063, "drizzle" },
                { 1150, "drizzle" },
                { 1153, "drizzle" },
                { 1168, "sleet" },
                { 1171, "sleet" },

                // Hujan
                { 1180, "rain" },
                { 1183, "rain" },
                { 1186, "rain" },
                { 1189, "rain" },
                { 1192, "rain" },
                { 1195, "rain" },
                { 1198, "sleet" },
                { 1201, "sleet" },
                { 1240, "rain" },
                { 1243, "rain" },
                { 1246, "rain" },

                // Salju
                { 1066, "snow" },
                { 1069, "sleet" },
                { 1072, "sleet" },
                { 1114, "snow" },
                { 1117, "snow" },
                { 1204, "sleet" },
                { 1207, "sleet" },
                { 1210, "snow" },
                { 1213, "snow" },
```

```
                { 1216, "snow" },
                { 1219, "snow" },
                { 1222, "snow" },
                { 1225, "snow" },
                { 1237, "hail" },
                { 1249, "sleet" },
                { 1252, "sleet" },
                { 1255, "snow" },
                { 1258, "snow" },
                { 1261, "hail" },
                { 1264, "hail" },

                // Badai Petir
                { 1087, "thunderstorms-day" },
                { 1273, "thunderstorms-day-rain" },
                { 1276, "thunderstorms-rain" },
                { 1279, "thunderstorms-day-snow" },
                { 1282, "thunderstorms-snow" }
            };
        }

        // Fungsi untuk memuat icon cuaca berdasarkan kode cuaca dan waktu (siang/malam)
        public static Image LoadWeatherIcon(int weatherCode, bool isDay)
        {
            try
            {
                // Dapatkan nama icon berdasarkan kode cuaca dan waktu
                string iconName = GetIconName(weatherCode, isDay);
                // Gabungkan path folder dengan nama file icon
                string iconPath = Path.Combine(iconFolder, $"{iconName}.png");

                // Cek apakah file icon ada
                if (File.Exists(iconPath))
                {
                    // Buka file icon dan buat copy bitmap-nya
                    using (var originalImage = Image.FromFile(iconPath))
                    {
                        return new Bitmap(originalImage);
                    }
                }
                else
                {
                    // Tulis pesan debug jika icon tidak ditemukan
                    System.Diagnostics.Debug.WriteLine($"Icon not found: {iconPath}");
                    return null!;
                }
            }
            catch (Exception ex)
            {
                // Tangkap error dan tulis pesan debug
                System.Diagnostics.Debug.WriteLine($"Error loading icon: {ex.Message}");
                return null!;
            }
        }

        // Fungsi untuk memuat icon cuaca dengan ukuran custom (resize)
        public static Image LoadWeatherIcon(int weatherCode, bool isDay, int width, int height)
        {
            try
            {
                // Muat icon asli terlebih dahulu
                Image originalIcon = LoadWeatherIcon(weatherCode, isDay);
                if (originalIcon == null) return null!;

                // Buat bitmap baru dengan ukuran yang diinginkan
                Bitmap resizedIcon = new Bitmap(width, height);
                using (Graphics g = Graphics.FromImage(resizedIcon))
                {
                    // Gunakan interpolasi berkualitas tinggi untuk hasil resize yang bagus
                    g.InterpolationMode = System.Drawing.Drawing2D.InterpolationMode.HighQualityBicubic;
                    // Gambar icon asli ke bitmap baru dengan ukuran baru
                    g.DrawImage(originalIcon, 0, 0, width, height);
                }

                // Hapus icon asli dari memory untuk menghemat resource
                originalIcon.Dispose();
                return resizedIcon;
            }
            catch (Exception ex)
            {
                // Tangkap error dan tulis pesan debug
                System.Diagnostics.Debug.WriteLine($"Error resizing icon: {ex.Message}");
```

```
            return null!;
        }
    }

    // Fungsi private untuk mendapatkan nama icon berdasarkan kode cuaca dan waktu
    private static string GetIconName(int weatherCode, bool isDay)
    {
        string iconName;

        // Cari nama icon dari dictionary berdasarkan kode cuaca
        if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
        {
            // Jika kode cuaca tidak ditemukan, gunakan icon default
            System.Diagnostics.Debug.WriteLine($"⚠️ Weather code {weatherCode} NOT FOUND in mapping! Using default.");
            iconName = isDay ? "clear-day" : "clear-night";
        }
        else
        {
            // Tulis pesan debug jika mapping berhasil ditemukan
            System.Diagnostics.Debug.WriteLine($"✓ Weather code {weatherCode} mapped to: {iconName}");
        }

        // Ganti icon siang dengan icon malam jika waktu adalah malam
        if (!isDay)
        {
            if (weatherCode == 1000) // Cerah
            {
                iconName = "clear-night";
            }
            else if (weatherCode == 1003) // Cerah Berawan
            {
                iconName = "partly-cloudy-night";
            }
            else if (weatherCode == 1087) // Badai Petir
            {
                iconName = "thunderstorms-night";
            }
            else if (weatherCode == 1273) // Badai Petir dengan Hujan
            {
                iconName = "thunderstorms-night-rain";
            }
            else if (weatherCode == 1279) // Badai Petir dengan Salju
            {
                iconName = "thunderstorms-night-snow";
            }
            else if (iconName.Contains("-day"))
            {
                // Ganti semua icon yang mengandung "-day" menjadi "-night"
                iconName = iconName.Replace("-day", "-night");
            }
        }

        return iconName;
    }

    // Fungsi untuk mendapatkan daftar icon yang diperlukan untuk didownload
    public static List<string> GetRequiredIcons()
    {
        return new List<string>
        {
            // Icon untuk siang hari
            "clear-day.png",
            "partly-cloudy-day.png",
            "cloudy.png",
            "overcast.png",
            "mist.png",
            "fog.png",
            "drizzle.png",
            "rain.png",
            "snow.png",
            "sleet.png",
            "hail.png",
            "thunderstorms-day.png",
            "thunderstorms-day-rain.png",
            "thunderstorms-day-snow.png",
            "thunderstorms-rain.png",
            "thunderstorms-snow.png",

            // Icon untuk malam hari
            "clear-night.png",
            "partly-cloudy-night.png",
            "thunderstorms-night.png",
            "thunderstorms-night-rain.png"
```

```
                "thunderstorms-night-snow.png"
            };
        }

        // Fungsi untuk mengecek apakah icon tersedia di folder
        public static bool IsIconAvailable(int weatherCode, bool isDay)
        {
            // Dapatkan nama icon
            string iconName = GetIconName(weatherCode, isDay);
            // Gabungkan path folder dengan nama file
            string iconPath = Path.Combine(iconFolder, $"{iconName}.png");
            // Return true jika file ada, false jika tidak ada
            return File.Exists(iconPath);
        }

        // Fungsi untuk mendapatkan path icon berdasarkan deskripsi kondisi cuaca
        public static string GetIconPath(string condition)
        {
            // Ubah kondisi menjadi huruf kecil untuk pencocokan yang lebih mudah
            condition = condition.ToLower();
            string iconName = "not-available.png"; // icon default

            // Cek kondisi cuaca dan tentukan icon yang sesuai
            if (condition.Contains("sunny") || condition.Contains("clear"))
                iconName = "day-clear.png";
            else if (condition.Contains("cloud"))
                iconName = "cloudy.png";
            else if (condition.Contains("rain"))
                iconName = "rain.png";
            else if (condition.Contains("thunder"))
                iconName = "thunderstorms.png";
            else if (condition.Contains("snow"))
                iconName = "snow.png";
            else if (condition.Contains("mist") || condition.Contains("fog"))
                iconName = "fog.png";

            // Gabungkan path folder dengan nama icon
            return Path.Combine(Application.StartupPath, "Icons");
        }

        // Deskripsi cuaca dalam Bahasa Inggris
        public static string GetWeatherDescription(int weatherCode)
        {
            // Dictionary untuk menyimpan deskripsi cuaca dalam Bahasa Inggris
            var descriptions = new Dictionary<int, string>
            {
                { 1000, "Clear" },
                { 1003, "Partly Cloudy" },
                { 1006, "Cloudy" },
                { 1009, "Overcast" },
                { 1030, "Mist" },
                { 1063, "Possible Rain" },
                { 1135, "Fog" },
                { 1147, "Freezing Fog" },
                { 1150, "Light Drizzle" },
                { 1153, "Light Drizzle" },
                { 1180, "Light Rain" },
                { 1183, "Light Rain" },
                { 1186, "Moderate Rain" },
                { 1189, "Moderate Rain" },
                { 1192, "Heavy Rain" },
                { 1195, "Heavy Rain" },
                { 1087, "Thunderstorm" },
                { 1210, "Light Snow" },
                { 1213, "Light Snow" },
                { 1219, "Moderate Snow" },
                { 1225, "Heavy Snow" },
                { 1237, "Hail" },
                { 1273, "Thunder Rain" },
                { 1276, "Heavy Thunder Rain" }
            };

            // Return deskripsi jika kode cuaca ditemukan, jika tidak return "Unknown"
            return descriptions.ContainsKey(weatherCode)
                ? descriptions[weatherCode]
                : "Unknown";
        }
    }
}
```

5. Program.cs

```
using System;
using System.Windows.Forms;

namespace WeatherApp
{
    static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}

```

Screenshot Tampilan Aplikasi